UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL

FLUTTER



JULIO 2026-DICIEMBRE 2026

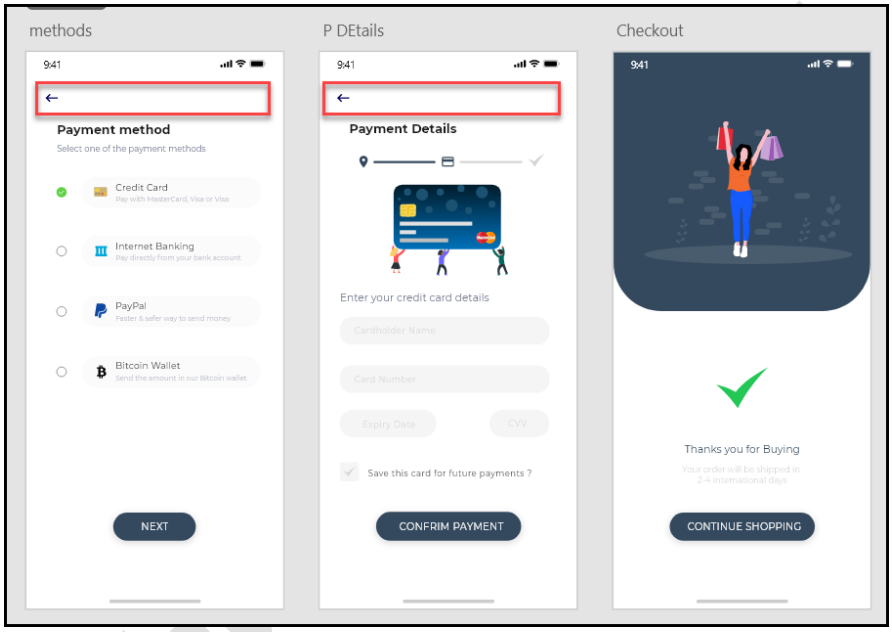
16/10/2026

Ejercicio # 1

En esta práctica vamos a continuar con el diseño de la primera pantalla para la aplicación en Flutter.

Métodos de pago #1

1. En esta práctica vamos a empezar con el diseño del **AppBar** para la primera pantalla de la aplicación, esto se muestra en el **mockup**:



2. En el mockup de la figura anterior se puede apreciar que el **AppBar** no es muy **notorio**, únicamente hay un **icono** en la parte superior de la **esquina izquierda**. Se puede visualizar una flecha en el **AppBar**. Las dos primeras pantallas tienen este icono en el **AppBar** y le tercera pantalla **no tiene este icono**.

Icono del AppBar del mockup.

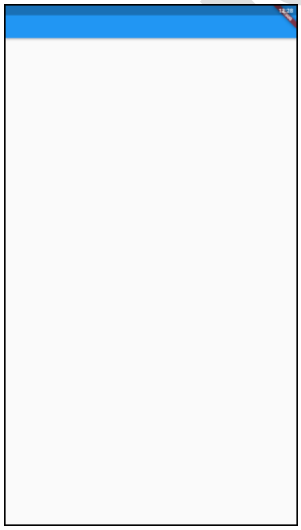


3. En el archivo **payment_method_screen.dart**, lo primero que **vamos** a realizar es agregar el **AppBar** en el **Scaffold** de la clase **PaymentMethodScreen**, esta clase representa la primera pantalla, pantalla/ventana/formulario para realizar el método de pago de la aplicación. El código sería el siguiente.



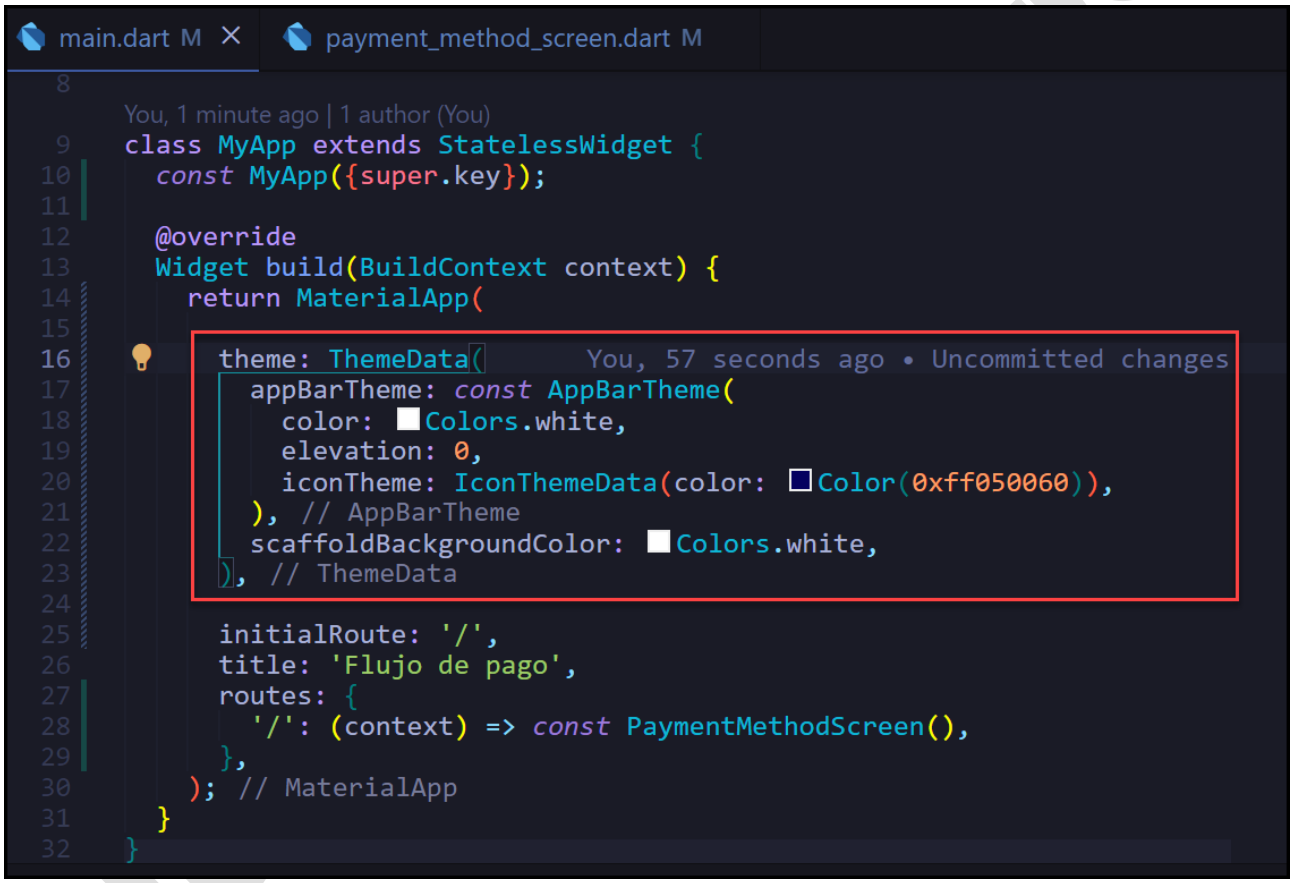
4. Note en la figura anterior que se ha **eliminado** del **Scaffold** la palabra reservada **const**, la palabra **const** en el **Scaffold** genera un error en el código porque el **AppBar** es dinámico y por lo tanto **no puede ser constante** (es decir no puede ser inmutable).

5. Si **ejecuta** el proyecto tenemos la siguiente salida.



6. Como se puede visualizar en la figura anterior, el **AppBar** es **azul** y **no blanco** que es lo que se requiere de acuerdo con el **mockup**, además el **AppBar** es el mismo en las 2 pantallas y la tercera pantalla no tiene **AppBar**, por lo tanto, lo mejor es poner el **AppBar** en el **tema del proyecto**, para que tengan el **AppBar** todas las pantallas del proyecto y con el mismo **formato** y **UI**.

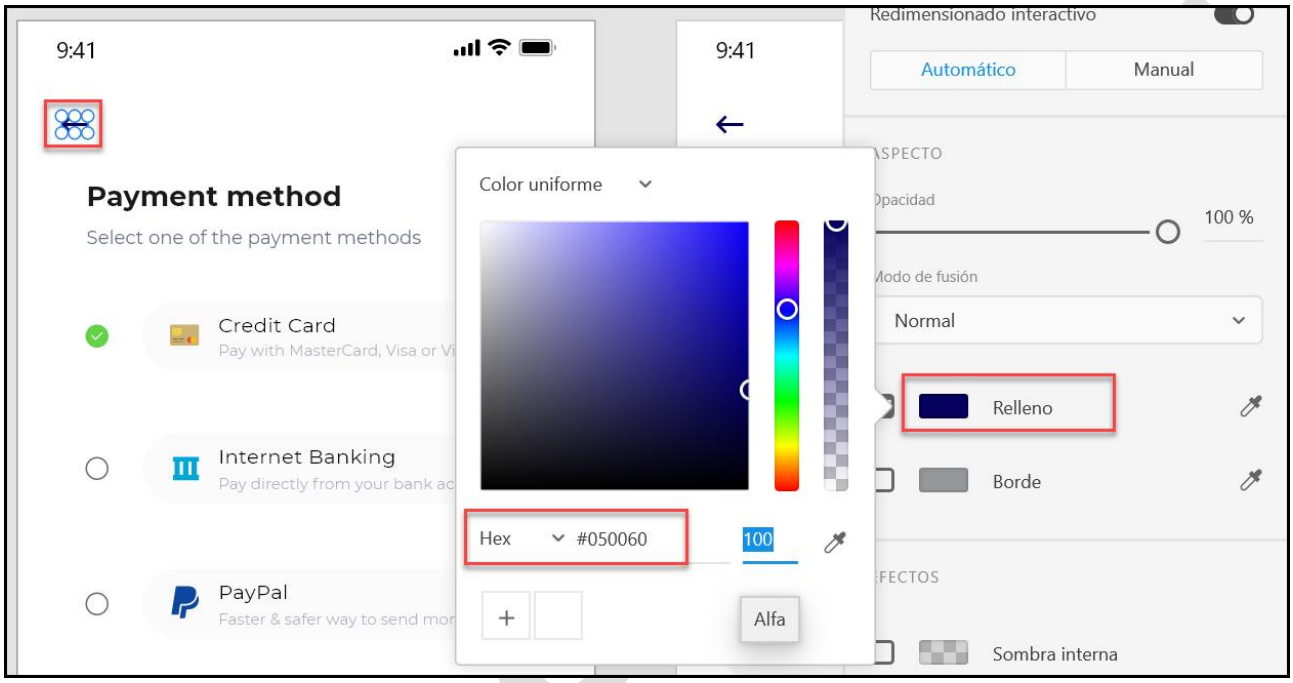
7. Por lo tanto **vaya** al archivo **main.dart** y en la clase **MyApp**, dentro del método **build** **digite** el código siguiente para establecer el **tema** del proyecto.



8. Para poner el **color específico** en Flutter se usa el siguiente formato, **primero** el valor: **0xff** (en minúsculas o mayúsculas) para cualquier color, y **luego** el valor del **color deseado en hexadecimal**, en este caso el valor en hexadecimal es **050060**, el valor completo para el color en Flutter seria **0xff050060**, que es el valor que está en el código de la figura anterior.

9. Para saber el **valor exacto** en **hexadecimal** del **color del icono** para el **AppBar** del mockup utilizamos **Adobe XD** y **procedemos** de la forma indicada en el **ejercicio 4** de la práctica **Payment (001)-Preparacion**. Esto seria se la siguiente forma.

Una vez **seleccionado** el icono del **AppBar** en **Adobe XD** podemos determinar el valor exacto del **color del botón** (flecha) para el **AppBar**.

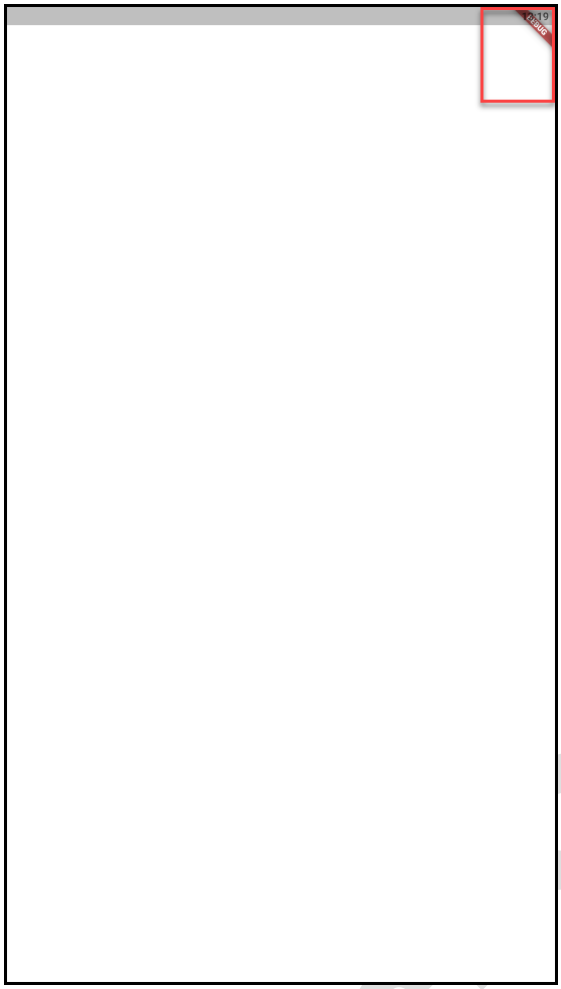


Botón (icono/fecha) del **AppBar** seleccionado.



10. Como se puede visualizar en la figura anterior, el **valor exacto** que tiene en el **mockup** en **hexadecimal** es #**050060**, para ubicarlo en Flutter **eliminamos** el carácter de **numeral** (#) y **agregamos** este número al prefijo **0xff**, y tendríamos el valor exacto para el color de un determinado componente, en este caso el valor para el color del **icono con una flecha hacia atrás** del **AppBar** seria **0xff050060**. El cual a primera vista parecería que es de color negro, pero como se pudo comprobar es una variante de este color.

11. **Ejecute** el proyecto.



12. Como se puede visualizar en la figura anterior, el **AppBar** ahora es **blanco** que es lo que se requiere según el **mockup**, pero aún falta el **icono** con la **flecha atrás**.

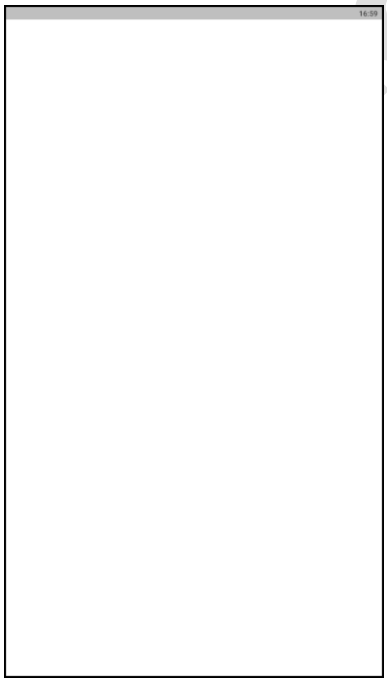
13. Para **eliminar** de la esquina superior derecha la palabra **DEBUG**, la cual muestra el **estado de la App**, lo cual indica que la App está en modo **debug**, en el archivo **main.dart** dentro del widget **MateriaApp** digite el siguiente código.

Flutter Clay Aldás

main.dart M × payment_method_screen.dart M

```
8
    You, 10 seconds ago | 1 author (You)
9   class MyApp extends StatelessWidget {
10    const MyApp({super.key});
11
12    @override
13    Widget build(BuildContext context) {
14      return MaterialApp(        You, 10 seconds ago • Uncommitted chang
15
16        debugShowCheckedModeBanner: false,
17
18        theme: ThemeData(
19          appBarTheme: const AppBarTheme(
20            color: Colors.white,
21            elevation: 0,
22            iconTheme: IconThemeData(color: Color(0xff050060)),
23          ), // AppBarTheme
24          scaffoldBackgroundColor: Colors.white,
25        ), // ThemeData
26
27        initialRoute: '/',
28        title: 'Flujo de pago',
29        routes: {
30          '/': (context) => const PaymentMethodScreen(),
31        },
32      ); // MaterialApp
```

14. Al **ejecutar** la aplicación la **salida** del programa ahora es el **siguiente**.



15. En la figura anterior se puede visualizar que **ya no se muestra** la palabra **DEBUG**.

16. En el archivo **payment_method_screen.dart**, vaya al **widget** (clase) **PaymentMethodScreen** que representa la **pantalla** (pantalla/ventana de inicio) y agregue a continuación el código para establecer el **icono** del **AppBar** por medio del parámetro con nombre **leading**. Para que el botón sea **clicleable** (habilitado, es decir se pueda **hacer un clic**), se debe usar el parámetro **onPressed** de la clase **IconButton**. El código se muestra a continuación.

payment_method_screen.dart

```dart
import 'package:flutter/material.dart';

class PaymentMethodScreen extends StatefulWidget {
  const PaymentMethodScreen({super.key});

  @override
  State<StatefulWidget> createState() {
    return PaymentMethodState();
  }
}

class PaymentMethodState extends State<PaymentMethodScreen> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(

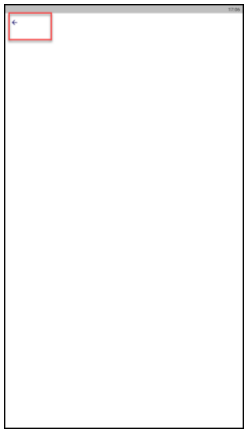
      appBar: AppBar(

        leading: IconButton(
          icon: const Icon(Icons.arrow_back),
          onPressed: () {},
        ), // IconButton

      ), // AppBar
    ); // Scaffold
  }
}

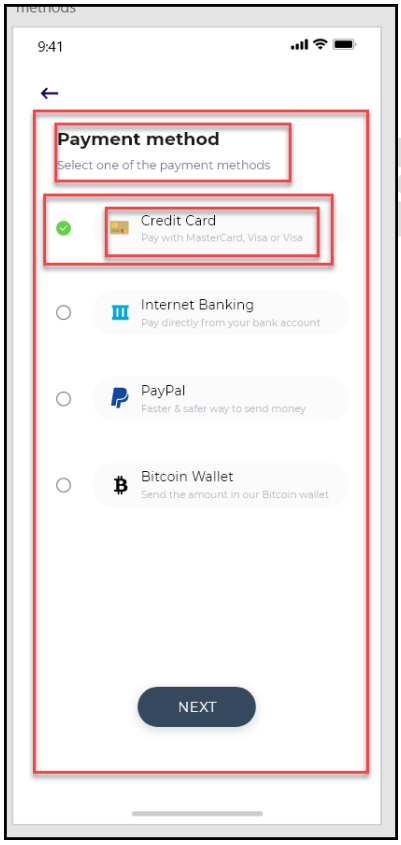
```

17. Si **ejecuta** el código puede visualizar el **icono** y lo podemos **cliclear** (al presionar sobre este icono se hace un splash), aunque por el momento el botón no hace nada.



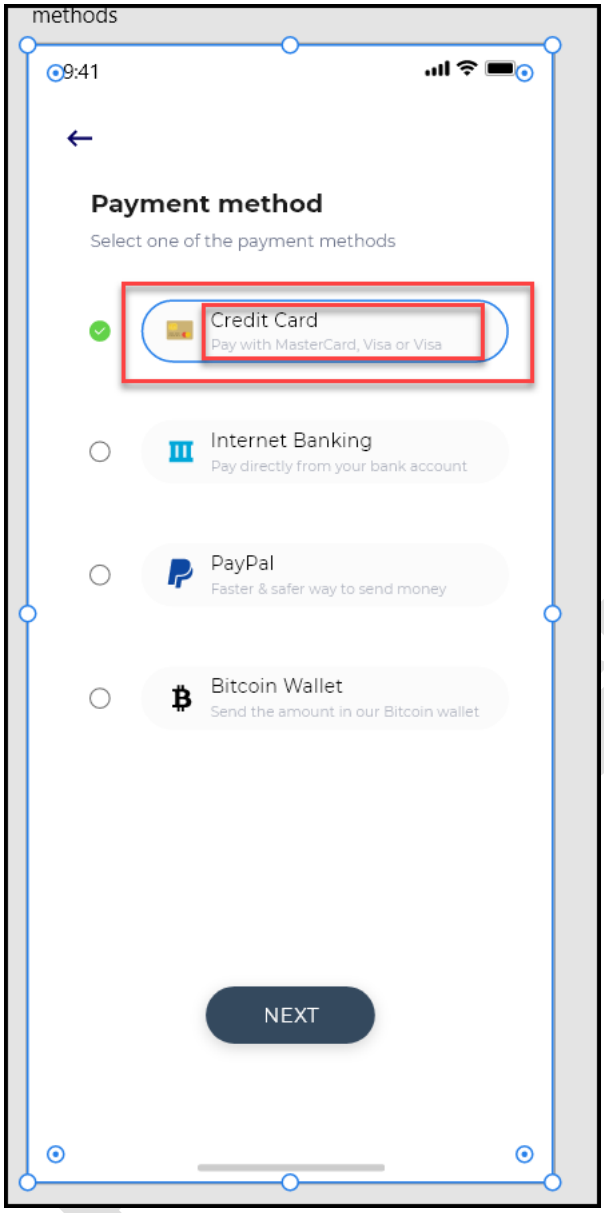
18. Ya tenemos la primera parte del diseño de la ventana.

19. Vamos a analizar los **diferentes widgets** que tiene la primera ventana/pantalla del **mockup** para luego crearlos en **Flutter**. Si **estudiamos** la siguiente parte de la **pantalla** del **mockup**, podemos deducir que se necesita una **columna** (widget **Column**), dentro de esta tenemos **varias filas** (widgets **Row**), a su vez dentro de las primeras filas (Row) tenemos otra **columna** (Column) con otra fila (Row) que esta alineada (centrada).



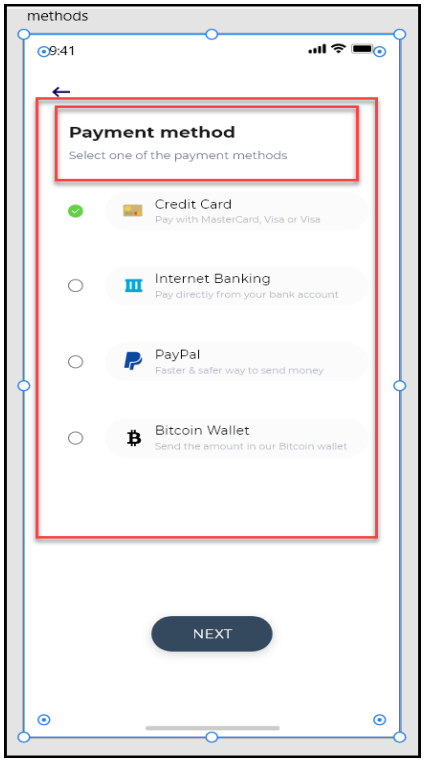
Clay Aldás

20. Además, si visualiza el mockup en **Adobe XD** puede notar que las **filas** más internas mostradas en la **figura** a continuación, en el borde de estas tienen un **radio**, esta **fila** (Row) dentro tiene otra **fila** (Row) que contiene una **imagen** (Image) y una **columna** (Column) con **2 textos** (Text), este análisis se muestra a continuación.



21. Por lo tanto, vamos a proceder con el **diseño** en **Flutter** de esta parte analizada anteriormente.

22. Según el mockup, primero vamos a **agregar** los **2 textos** (Text) que están en la parte **superior** de la pantalla. Esto se muestra a continuación en el **mockup**.



23. En el archivo **payment_method_screen.dart** en **Flutter**, en la clase **PaymentMethodScreen**, en el **Scaffold** luego del **AppBar**, agregue el parámetro **body**, en este parámetro este digite el siguiente código.

```dart
payment_method_screen.dart  main.dart

  @override
  State<StatefulWidget> createState() {
    return PaymentMethodState();
  }
}

class PaymentMethodState extends State<PaymentMethodScreen> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        leading: IconButton(
          icon: const Icon(Icons.arrow_back),
          onPressed: () {},
        ),          // IconButton
      ), // AppBar

      body: const Column(
        children: <Widget> [
          Text('Método de pago'),
          Text('Seleccione uno de los métodos de pago')
        ], // <Widget>[]
      ), // Column

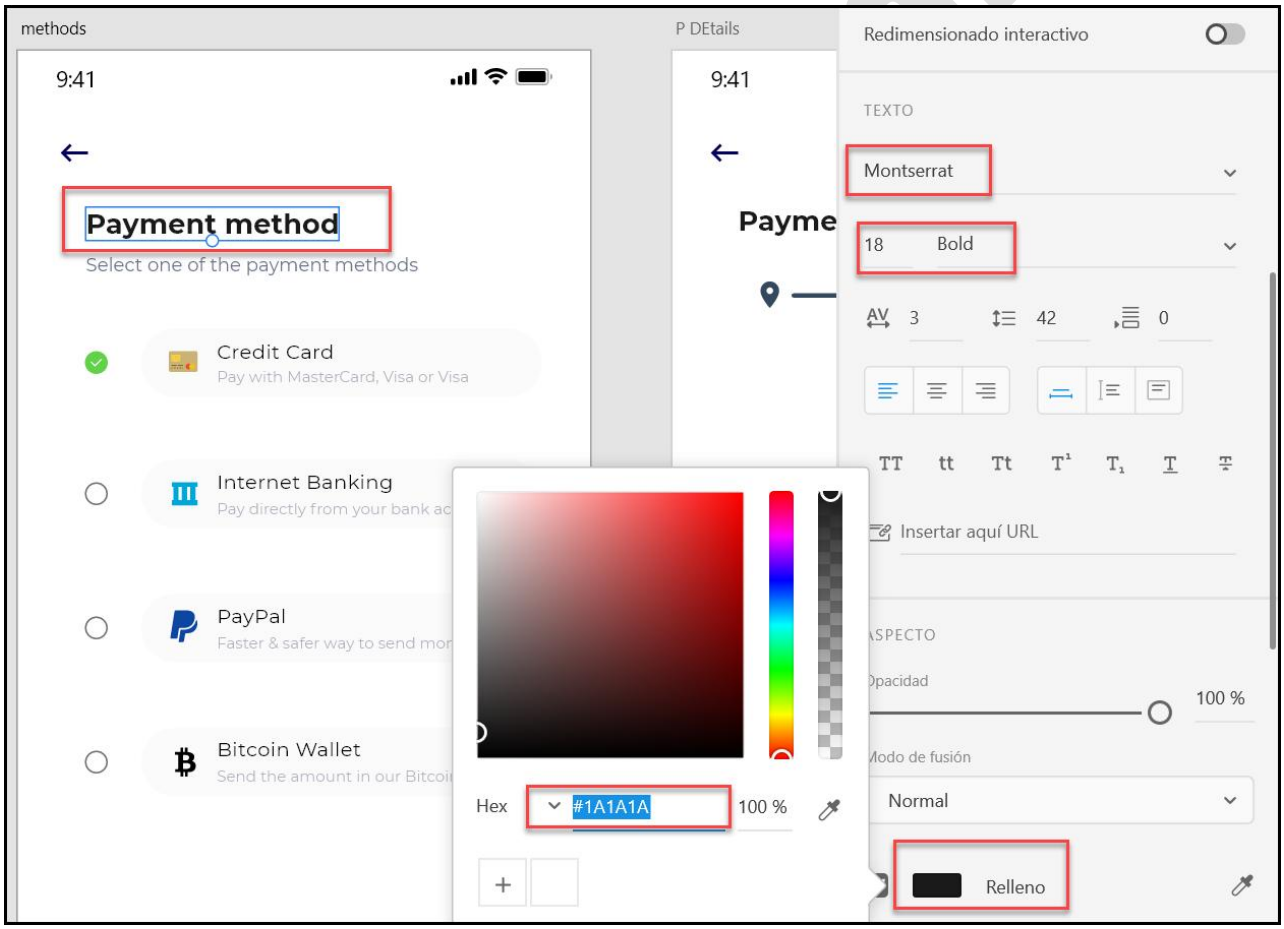

    ); // Scaffold
  }
}
```

24. Ahora nos **ayudamos** del mockup y en **Adobe XD** podemos analizar las siguientes **características** del **estilo** de los **textos** para realizar la **pantalla** lo más parecido posible al **mockup**.

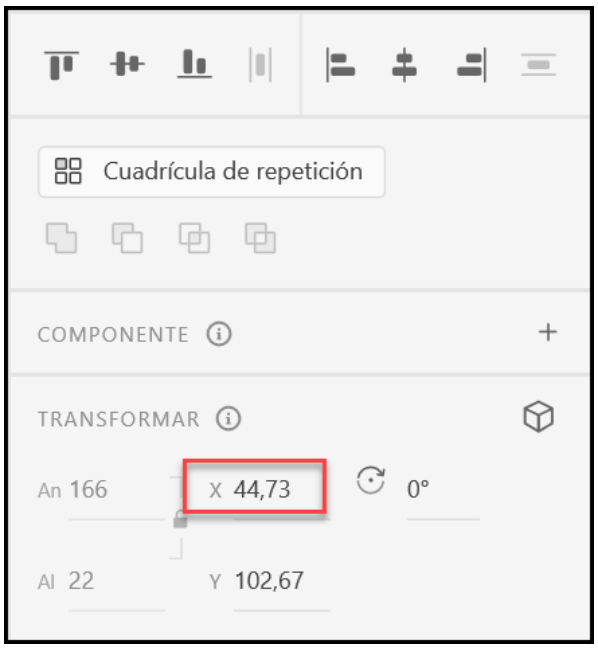
Primer texto a crear:



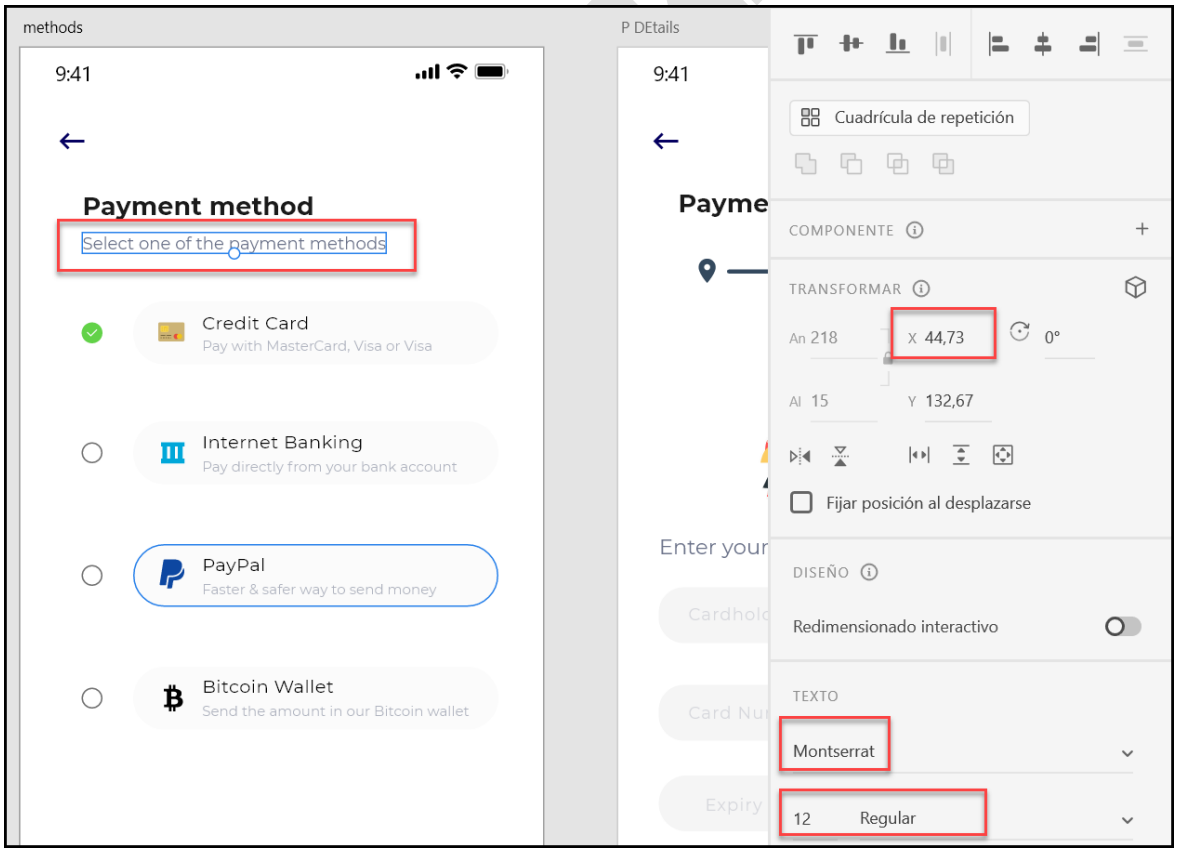
En **Adobe XD** se muestra varias características del **texto** seleccionado (**Payment method**).



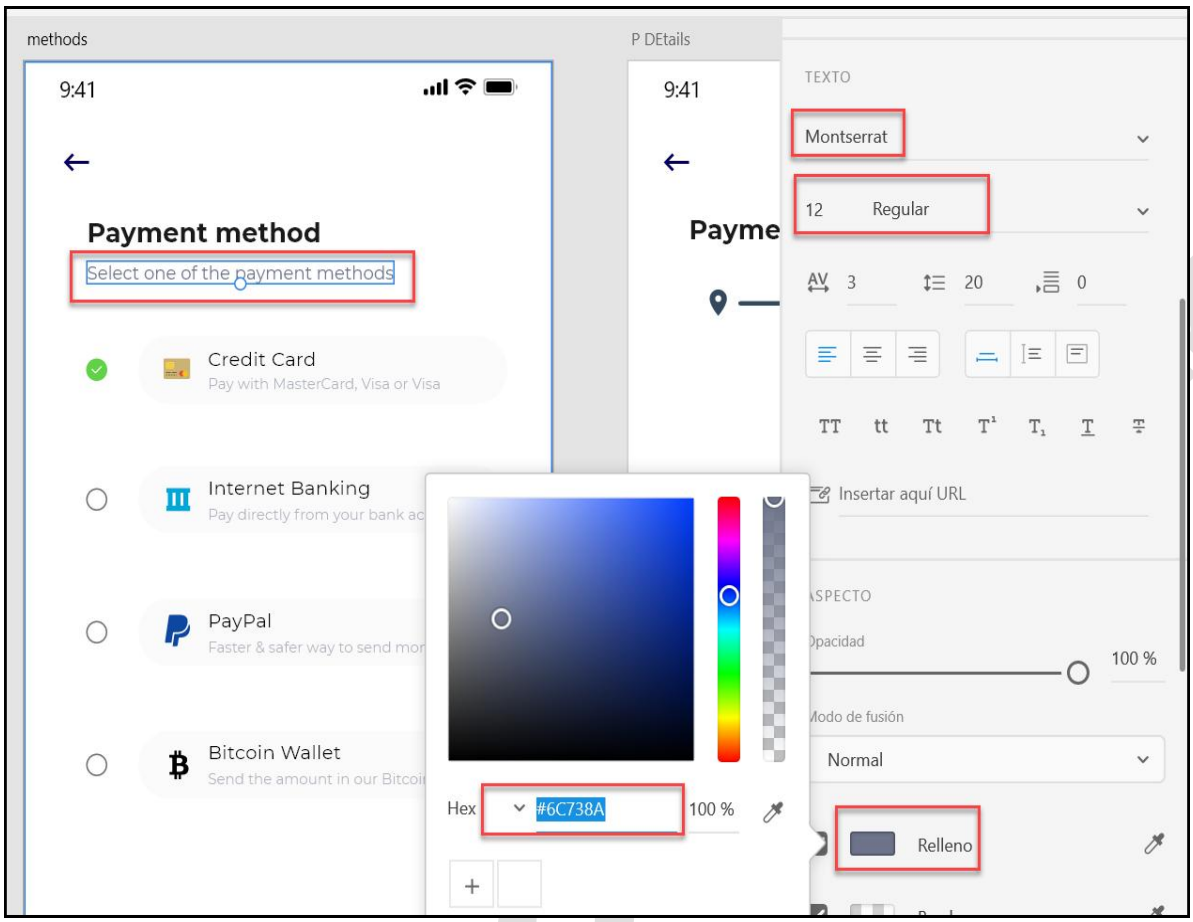
Ubicación del **texto** y también **ancho** y **alto**.



Segundo texto del mockup seleccionado:



Se muestra en **Adobe XD** varias propiedades del **segundo texto** seleccionado.



De las **figuras anteriores** analizadas en **Adobe XD** podemos obtener por ejemplo los siguientes datos.

- Los dos textos están **alineados** a la izquierda.
- El **primer texto** está en **negrita**, el **tamaño** del texto es de **18** puntos, la **fuente** es Monserrat, el **color** del texto es **#1A1A1A**,
- Para el **segundo texto** el tipo de texto es **regular**, el **tamaño** es de **12** puntos, la **fuente** es Monserrat, el **color** es **#6C738A**.
- Los **2 textos** tienen un **padding** de **44,73** (vamos a trabajar con 45 en Flutter) para todos los valores (izquierda, derecha, arriba, abajo).

25. El **código** en Flutter para el análisis realizado anteriormente se muestra a continuación. Por lo tanto, en el archivo **payment_method_screen.dart**, en la clase **PaymentMethodScreen**, el código debe ir dentro de la propiedad **body** del **scaffold** para que se muestre en la pantalla.

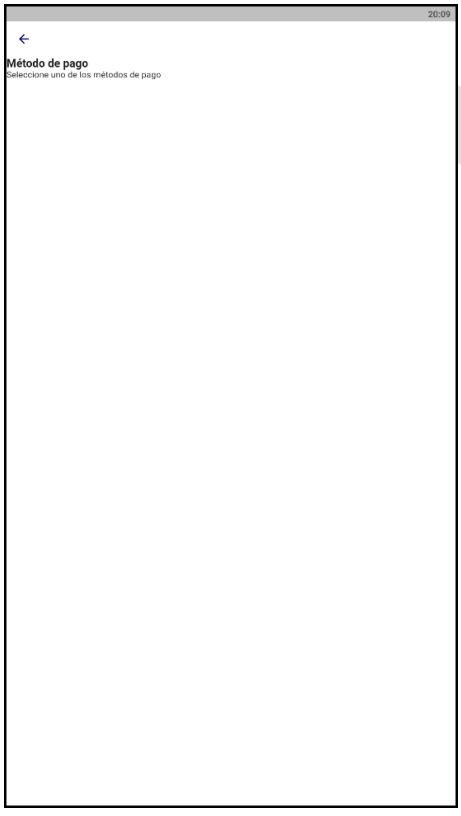
payment_method_screen.dart

```
      leading: IconButton(
        icon: const Icon(Icons.arrow_back),
        onPressed: () {},
      ), // IconButton
    ), // AppBar

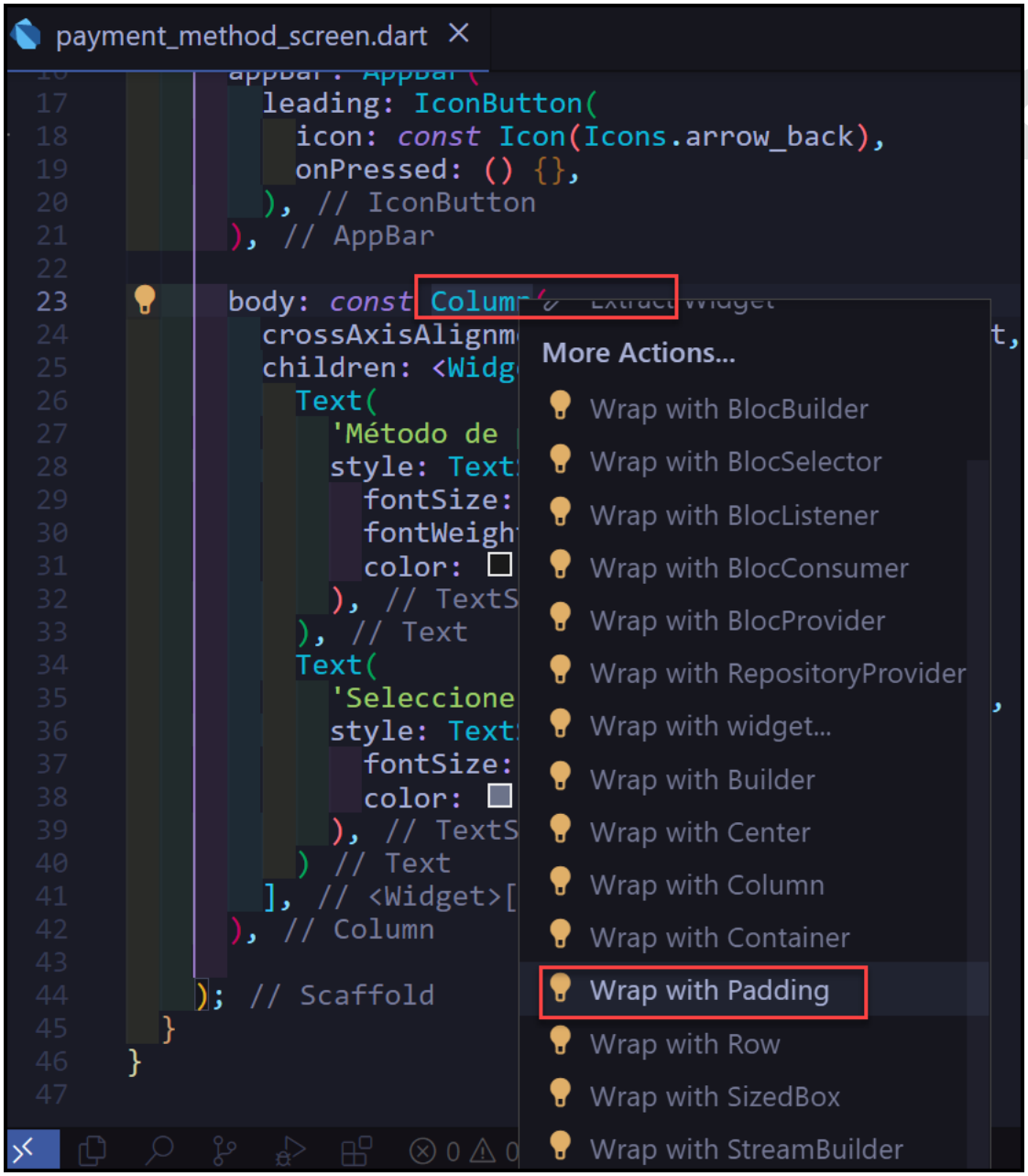
    body: const Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: <Widget>[
        Text(
          'Método de pago',
          style: TextStyle(
            fontSize: 18,
            fontWeight: FontWeight.bold,
            color: Color(0xff1A1A1A),
          ), // TextStyle
        ), // Text
        Text(
          'Seleccione uno de los métodos de pago',
          style: TextStyle(
            fontSize: 12,
            color: Color(0xff6C738A),
          ), // TextStyle
        ) // Text
      ], // <Widget>[]
    ), // Column

  ); // Scaffold
 }
}
```

26. Al **ejecutar** el proyecto tenemos la siguiente pantalla.



27. Vamos a agregar bordes alrededor del **Column** con un **padding**, el **padding** debe cubrir toda la **columna**, para eso nos ayudarnos del **Visual Studio Code**. Ponga el mouse **sobre** la palabra **Column**, presione las teclas **CONTROL + PUNTO** (CTRL + .), de la ventana **emergente seleccione** la opción **Wrap with Padding**, esto permite **envolver** el widget **Column** señalado con el mouse con un widget **Padding**, de la siguiente forma.



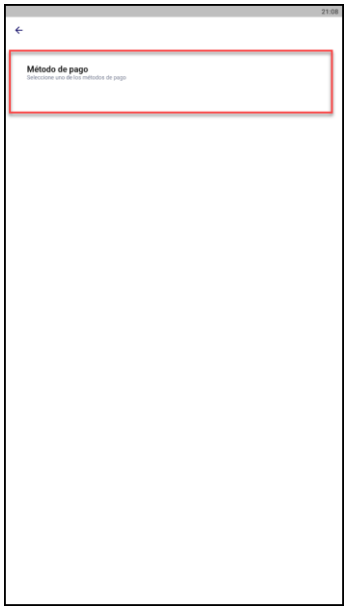
28. Luego del código generado con el asistente agregue un la propiedad **padding** de 45 pixeles en todos los lados (izquierda, derecha, arriba y abajo), es el código es el siguiente.

main.dart M payment_method_screen.dart M

```
21        ), // AppBar
22
23        body: const Padding(      You, 2 seconds ago • Uncommitted changes
24
25          padding: EdgeInsets.all(45.0),
26
27          child: Column(
28
29            crossAxisAlignment: CrossAxisAlignment.start,
30            children: <Widget>[
31              Text(
32                'Método de pago',
33                style: TextStyle(
34                    fontSize: 18,
35                    fontWeight: FontWeight.bold,
36                    color: Color(0Xff1A1A1A)), // TextStyle
37              ), // Text
38              Text(
39                'Seleccione uno de los métodos de pago',
40                style: TextStyle(
41                  fontSize: 12,
42                  color: Color(0xff6C738A),
43                ), // TextStyle
44              ), // Text
45            ], // <Widget>[]
46          ), // Column
```

29. Para que no haya **errores** ni **warnings** en el código anterior, tome en cuenta que ahora el **padding** tiene la palabra **const** y el widget **Column** ya no lo tiene, **analice el porqué** de la lógica de esto.

30. Al **ejecutar** el programa tenemos la siguiente salida en pantalla.



31. Para **establecer** la fuente (**font)** de la pantalla lo mejor es hacerlo en el **tema** (**theme**) puesto que el **font** es el mismo **para todas las pantallas**. Por lo tanto **vamos** al archivo **main.dart** y en el parámetro **theme** de la clase **MaterialApp**, digite el código indicado a continuación.

```dart
main.dart

Run | Debug | Profile
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

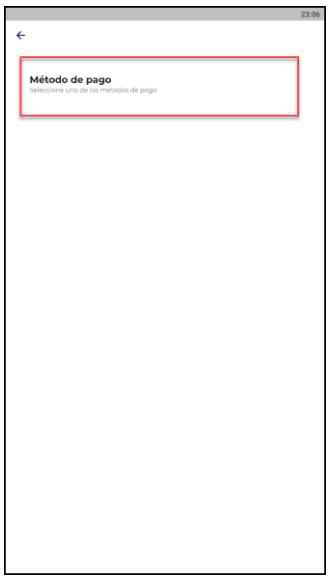
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,

      theme: ThemeData(

        fontFamily: 'Montserrat',

        appBarTheme: const AppBarTheme(
          color: Colors.white,
          elevation: 0,
          iconTheme: IconThemeData(color: Color(0xff050060)),
        ), // AppBarTheme
        scaffoldBackgroundColor: Colors.white,
      ), // ThemeData
      initialRoute: '/',
      title: 'Flujo de pago',
      routes: {
        '/': (context) => const PaymentMethodScreen(),
      },
    ); // MaterialApp
  }
```

32. Al **ejecutar** el proyecto se puede visualizar el cambio en la **fuente**.



Nota	• Hay momentos donde la característica de **hot reload** no funciona en el IDE de **Visual Studio Code**. Por lo tanto, si los cambios realizados no se reflejan, debe **detener** el proyecto y **ejecutarlo** nuevamente.

Lecturas útiles para revisar y profundizar en los temas abordados en esta práctica.

AppBar class

https://api.flutter.dev/flutter/material/AppBar-class.html

ThemeData class

https://api.flutter.dev/flutter/material/ThemeData-class.html

Theme class

https://api.flutter.dev/flutter/material/Theme-class.html

IconButton class

https://api.flutter.dev/flutter/material/IconButton-class.html

Text class

https://api.flutter.dev/flutter/widgets/Text-class.html

Flutter-C y Aldás

Column class

https://api.flutter.dev/flutter/widgets/Column-class.html

TextStyle class

https://api.flutter.dev/flutter/painting/TextStyle-class.html

Color class

https://api.flutter.dev/flutter/dart-ui/Color-class.html

Padding class

https://api.flutter.dev/flutter/widgets/Padding-class.html

EdgeInsets class

https://api.flutter.dev/flutter/painting/EdgeInsets-class.html

CrossAxisAlignment enum

https://api.flutter.dev/flutter/material/material-library.html

FlutterClay Aldás

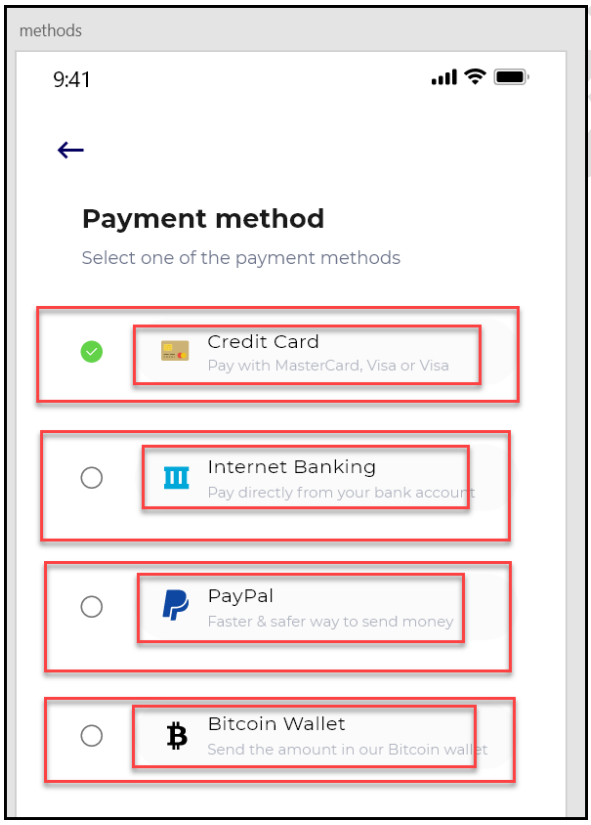
16/10/2026

Ejercicio # 2

En esta práctica vamos a continuar con el diseño de la primera pantalla para el método de pago en Flutter.

Métodos de pago #2

1. Vamos a seguir trabajando con la pantalla para los **métodos de pago**. Si vemos el **mockup** mostrado a continuación, se puede abordar el desarrollo de la interfaz de usuario de esta sección **escribiendo el código** para cada uno de los métodos de pago (Credit Card, Internet Banking, PayPal, Bitcoin Wallet), una manera de hacerlo sería **repetir** el código para cada uno de los 4 métodos de pago, esta es una forma válida de resolver la tarea, aunque se generaría mucho **código repetitivo**, y por lo tanto no es el método más práctico. El **mockup** se presenta a continuación.



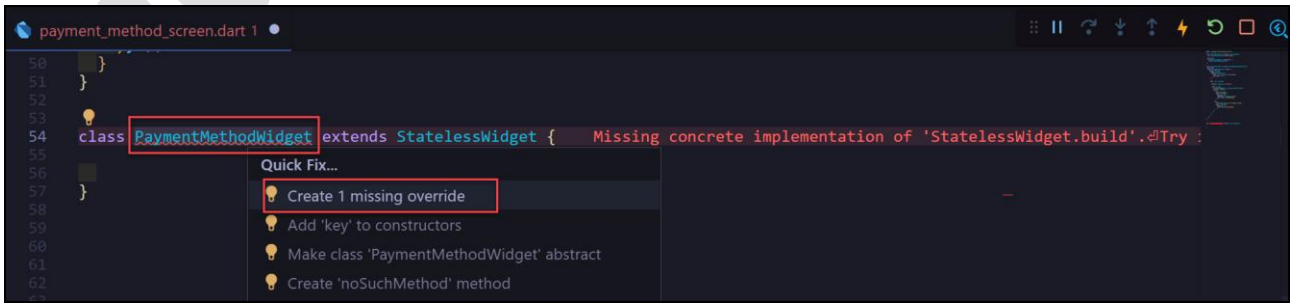
2. Si analizamos el **mockup** de mejor forma, podemos notar que **todos los métodos de pago** siguen el **mismo patrón** para la interfaz de usuario, por lo tanto, una mejor forma de realizar esta interfaz es **realizar un componente personalizado** (**widget**) que sea **reutilizable** para cada método de pago, y eslo que vamos a realizar en los siguientes apartados, lo que permitirá ahorrar líneas de código y **facilitar el mantenimiento**.

3. Por lo tanto, vaya al **final** del archivo **payment_method_screen.dart**, en este digite el código para crear el **widget personalizado** de tipo **StatelessWidget**, el nombre de este será **PaymentMethodWidget**, en cual se va realizar la **UI** (User Interface) para representar un **método de pago** (Credit Card, Internet Banking, PayPal, Bitcoin Wallet).

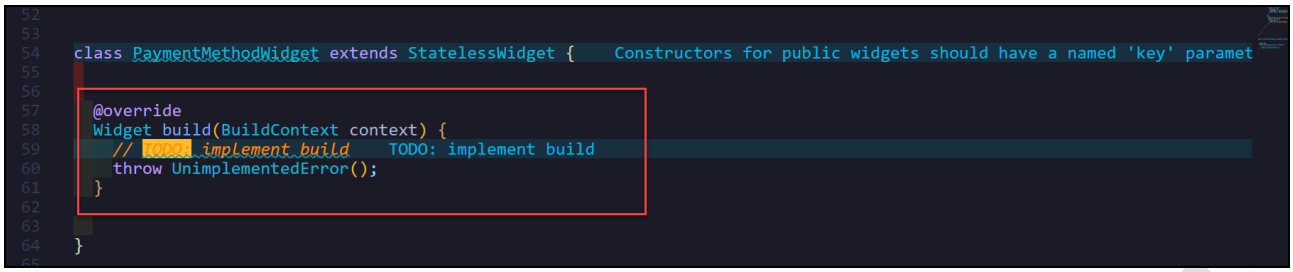


4. Una vez creada la clase, ponga el cursor en la clase **PaymentMethodWidget** y con la ayuda de Visual Studio Code **sobrescriba** el método **build**.

Una vez el cursor en la clase **PaymentMethodWidget** seleccione la opción indicada en la figura, **Create 1 missing override**.



Método **build** generado por el asistente.



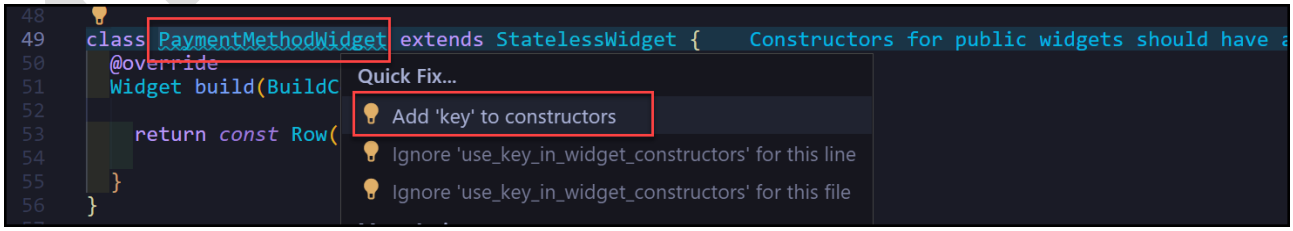
5. Borre el código generado para el método **build** y digite el siguiente código para regresar un **widget** de tipo **Row** el cual más adelante tendrá el método de pago.



6. En el código anterior el wigdet **Row** regresa un array de **widgets** con todo lo que contenga. Para el ejemplo, el método **build** va a regresar al diseño de la **UI** (User Interface) del **método de pago** de tal forma que sea un widget genérico (Para los métodos de pago: **Credit Card**, **Internet Banking**, **PayPal**, **Bitcoin Wallet**).

7. En la clase generada **PaymentMethodWidget**, **agregue** un **constructor** con el parámetro **key**. Use **Visual Studio Code** para generar este **constructor**, es decir presione CONTROL + PUNTO en la palabra **PaymentMethodWidget**, luego en la ventana emergente **seleccione** la opción **Add 'key' to constructors**.

Seleccione la opción indicada.



Se genera el **constructor** con el parámetro **key**.

```
class PaymentMethodWidget extends StatelessWidget {

  const PaymentMethodWidget({super.key});

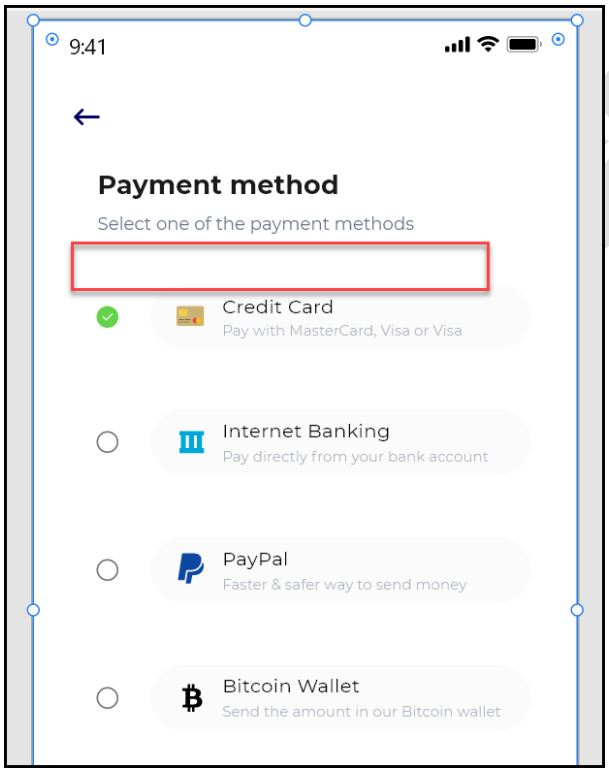
  @override
  Widget build(BuildContext context) {

    return const Row(children: <Widget> [],);

  }
}
```

8. Vamos al siguiente componente, de acuerdo con el **mockup** entre el segundo **texto** y el **primer tipo de pago** (Credit Card) hay un **espacio**, para esto vamos a utilizar un widget **SizedBox**.

Mockup con el **espacio en blanco** marcado en rojo.



9. Por lo tanto, en el archivo **payment_method_screen.dart** vaya a la clase **_PaymentMethodState** y digite el siguiente código para agregar el espaciado luego del **segundo Text**.

Widget **SizeBox** para agregar **espaciado vertical**.

payment_method_screen.dart

```
      body: const Padding(
        padding: EdgeInsets.all(45.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: <Widget>[
            Text(
              'Método de pago',
              style: TextStyle(
                fontSize: 18,
                fontWeight: FontWeight.bold,
                color: Color(0xff1A1A1A),
              ), // TextStyle
            ), // Text
            Text(
              'Seleccione uno de los métodos de pago',
              style: TextStyle(
                fontSize: 12,
                color: Color(0xff6C738A),
              ), // TextStyle
            ), // Text

            SizedBox( height: 20,),

          ], // <Widget>[]
        ), // Column
      ), // Padding
    ); // Scaffold
  }
}
```

10. **Luego** del **SizedBox** vamos a poner el **widget personalizado** que acabamos de crear para el método de pago (la clase **PaymentMethodWidget**), este representa el **un método de pago** (Credit Card), este método por el momento no tiene parámetros y aún no hace nada.

payment_method_screen.dart

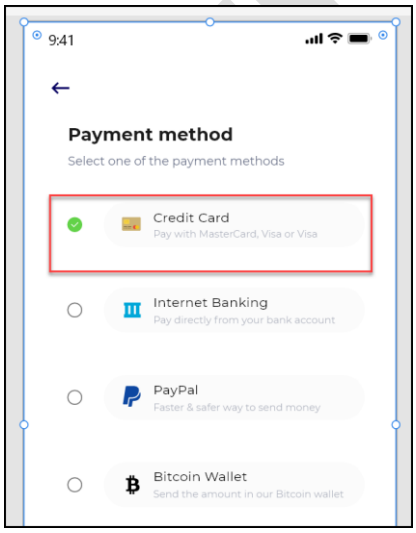
```dart
      body: const Padding(
        padding: EdgeInsets.all(45.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: <Widget>[
            Text(
              'Método de pago',
              style: TextStyle(
                fontSize: 18,
                fontWeight: FontWeight.bold,
                color: Color(0xff1A1A1A),
              ), // TextStyle
            ), // Text
            Text(
              'Seleccione uno de los métodos de pago',
              style: TextStyle(
                fontSize: 12,
                color: Color(0xff6C738A),
              ), // TextStyle
            ), // Text

            SizedBox( height: 20,),

            PaymentMethodWidget(),

          ], // <Widget>[]
        ), // Column
      ), // Padding
    ); // Scaffold
  }
}
```

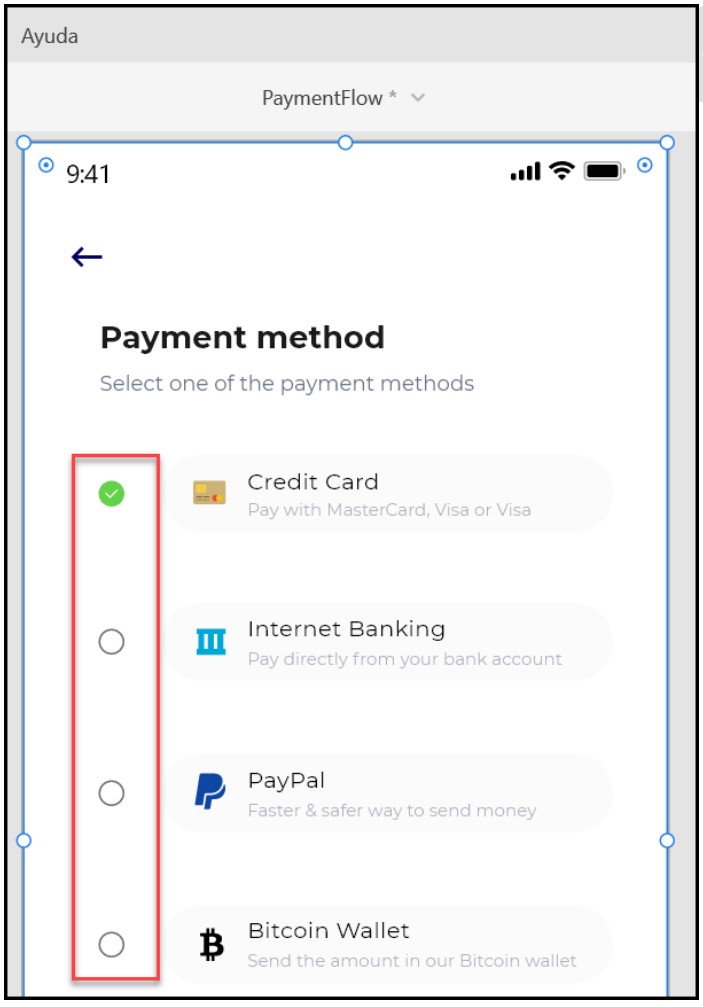
11. **Abra** nuevamente el mockup.



12. Si **analizamos** el **mockup** y el widget **PaymentMethodWidget** que dentro tiene una **fila** que es donde está el método de pago, podemos deducir que el método para que sea personalizable, **necesita 4 parámetros**, un parámetro para la **imagen de selección** de la opción del método de pago (este en el mockup anterior es un **radio button** en color **verde**), un parámetro para otra **imagen** que representa el **método de pago** (Credit Card, Internet Banking, PayPal, Bitcoin Wallet) y 2 **textos** (para título y subtítulo) que tienen la descripción del **método de pago**.

13. Para el **primer** parámetro podemos utilizar un **widget** cuyos valores sean **seleccionables**, por ejemplo, un widget de tipo **Radio** (RadioButton), pero nod hace falta enviar en el constructor de la clase el **primer parámetro** porque lo vamos a manejar de **forma interna en la clase**, por medio de un **icono** (verde) del paquete de **font awesome** y otro icono (**circulo)** cuando se seleccione el botón de **Radio** lo vamos a trabajar por medio de un **container**. Esto se muestra en la siguiente figura.

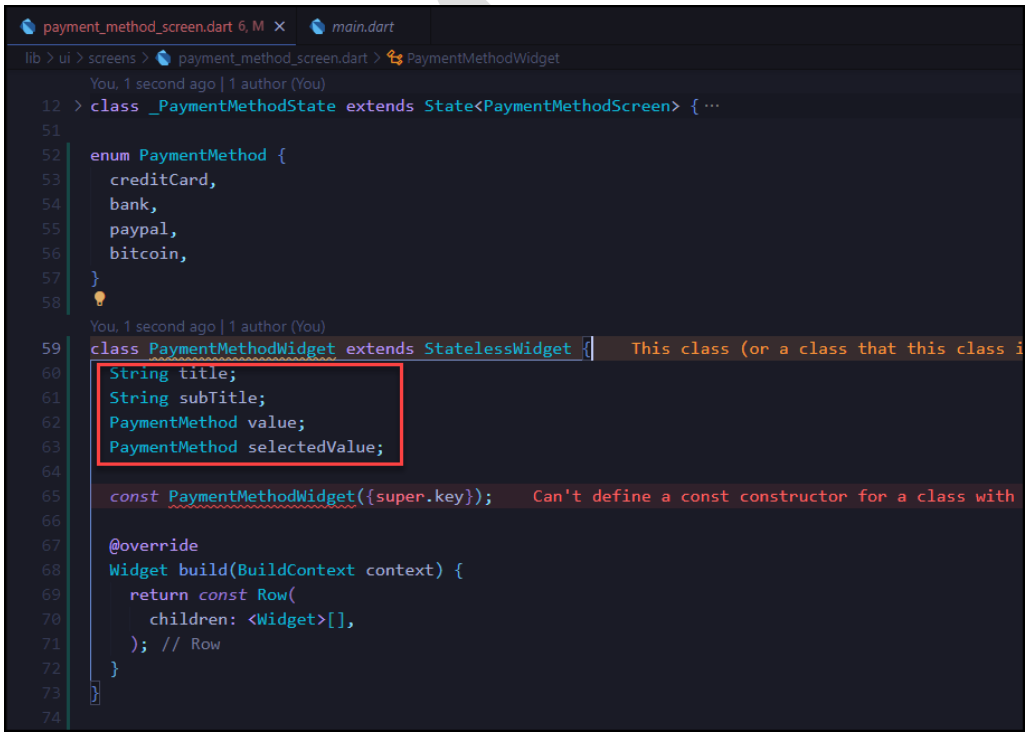
El mockup muestra la selección del **método de pago**, que puede ser tratada con un widget de tipo Radio en Flutter.



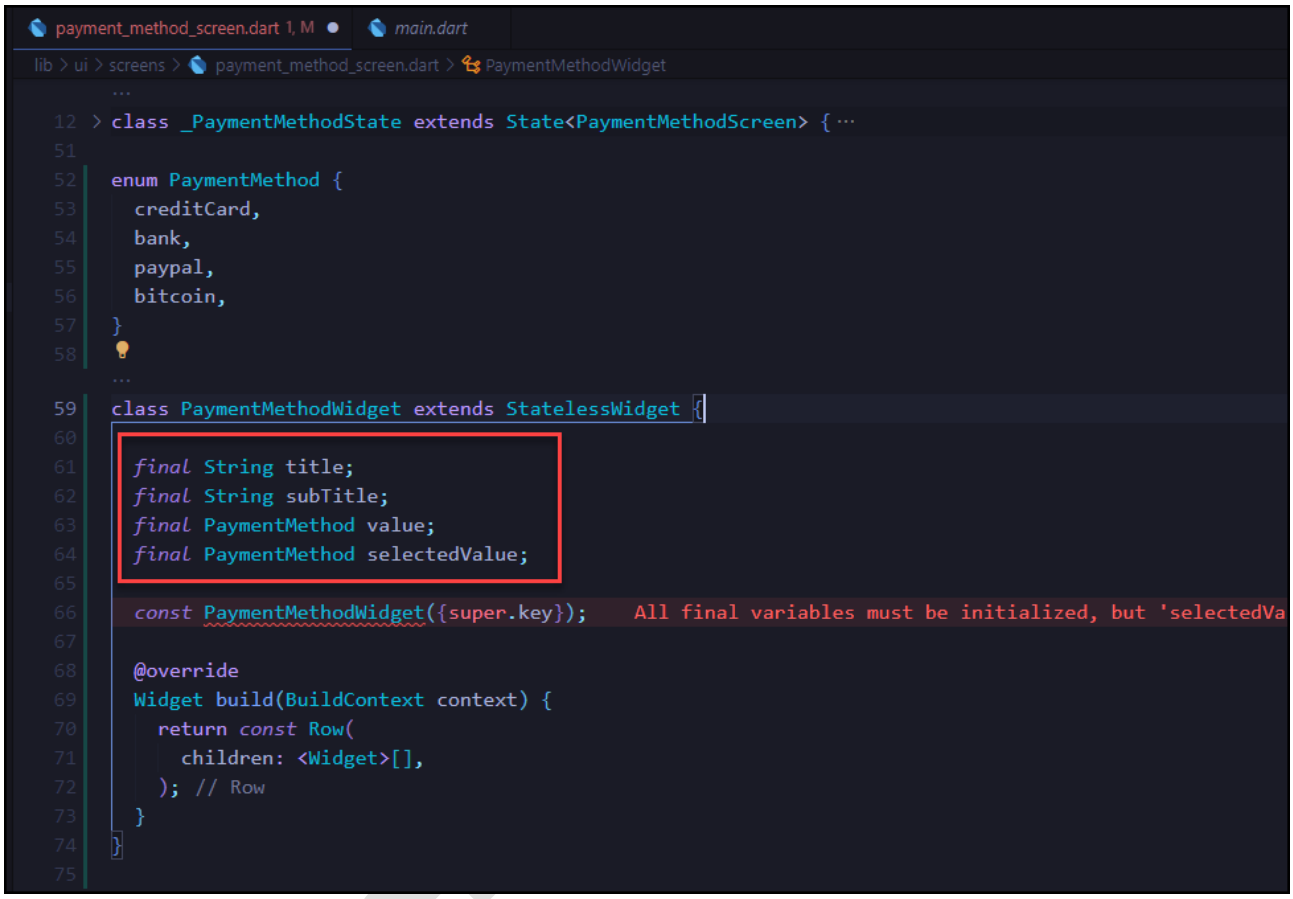
14., Para el **valor seleccionado** de entre diferentes tipos de pagos (Credit Card, Internet Banking, PayPal, Bitcoin Wallet) vamos a utilizar una **enumeración**, la **enumeración** es un tipo de dato y se debe definir fuera de las otras clases. Por lo tanto **cree** una enumeración y ponga como nombre **PaymentMethod**, esta enumeración se muestra a continuación y se lo va a crear en el archivo **payment_method_screen.dart,** antes de la clase **PaymentMethodWidget** por orden y fácil lectura, sin embargo también se lo puede poner en un archivo independiente. El código se lo muestra a continuación.



15. Ahora **dentro** del widget **PaymentMethodWidget** declare las 4 variables que van a representar los diferentes métodos de pago.

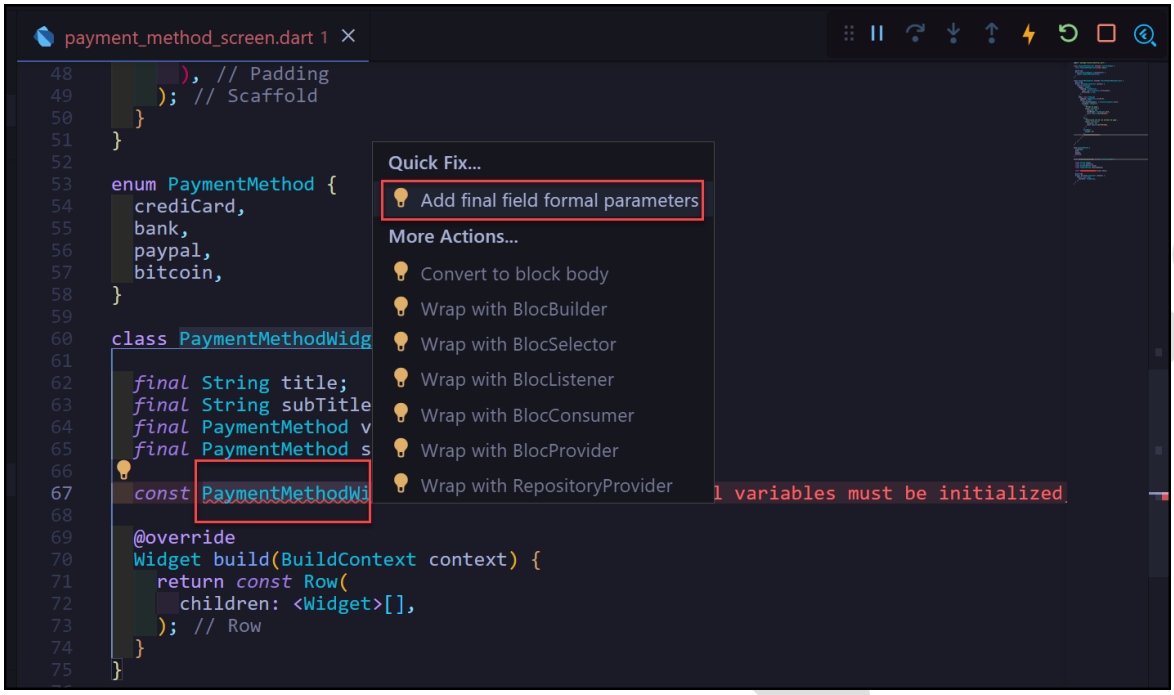


16. Como se puede visualizar en el código anterior, el **constructor** generar algunos **problemas**, hay varias formas de corregir estos problemas, para el ejemplo vamos a poner todos los datos de la clase como variables de tipo **final** puesto que una vez inicializadas las variables no van a ser modificadas es decir son **inmutables**.

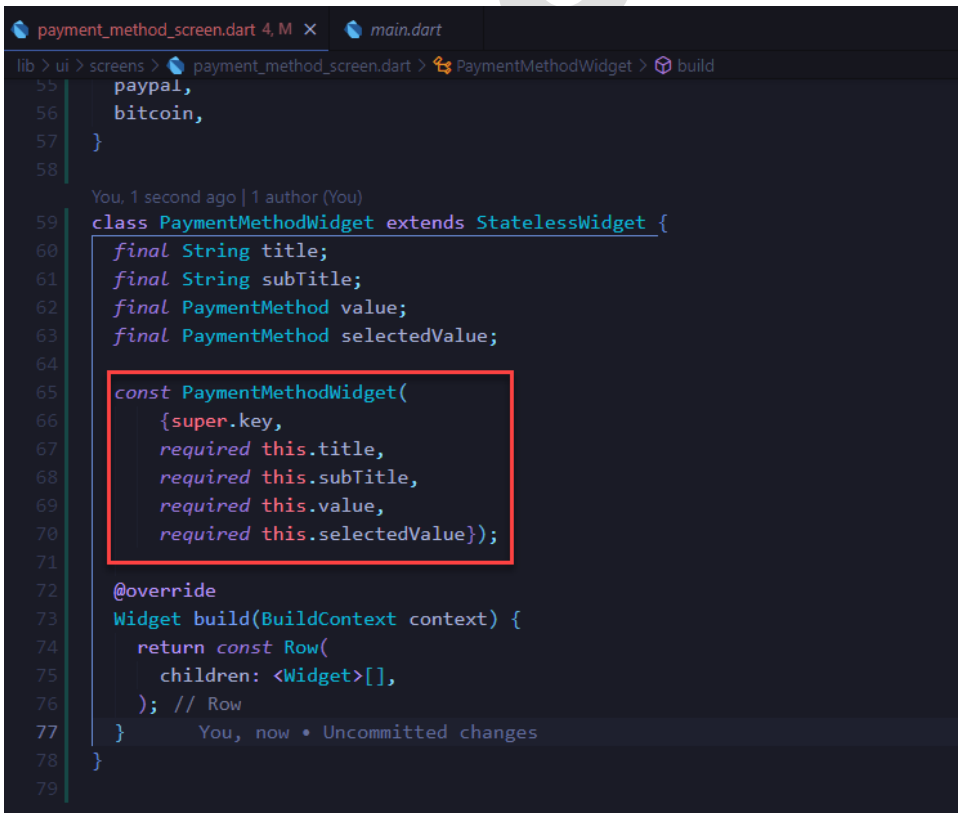


17. Ahora el IDE nos dice que todas las variables de tipo **final** deben ser **inicializadas**. Por estándar en **Flutter** las variables de tipo **final** deben ser **requeridas** (required), para el caso de ejemplo estos parámetros los vamos a crear como **parámetros por nombre** para que se más de trabajar y leer el código. El **primer parámetro** debe ser **obligatoriamente** el de nombre **key**, el cual es un parámetro **predeterminado** para identificar objetos del mismo tipo. Para modificar el constructor y agregar las variables al constructor, variables creadas anteriormente para la clase y que estas se generen **de forma correcta**, nos **ayudamos** del IDE de Visual Studio Code, para esto **ponemos el cursor en el nombre del constructor** (**PaymentMethodWidget**), luego presionamos las teclas **CONTROL + PUNTO** (CONTROL + .), de la ventana emergente seleccionamos la opción **Add final field formal parameters**, este se indica en la siguientes figuras.

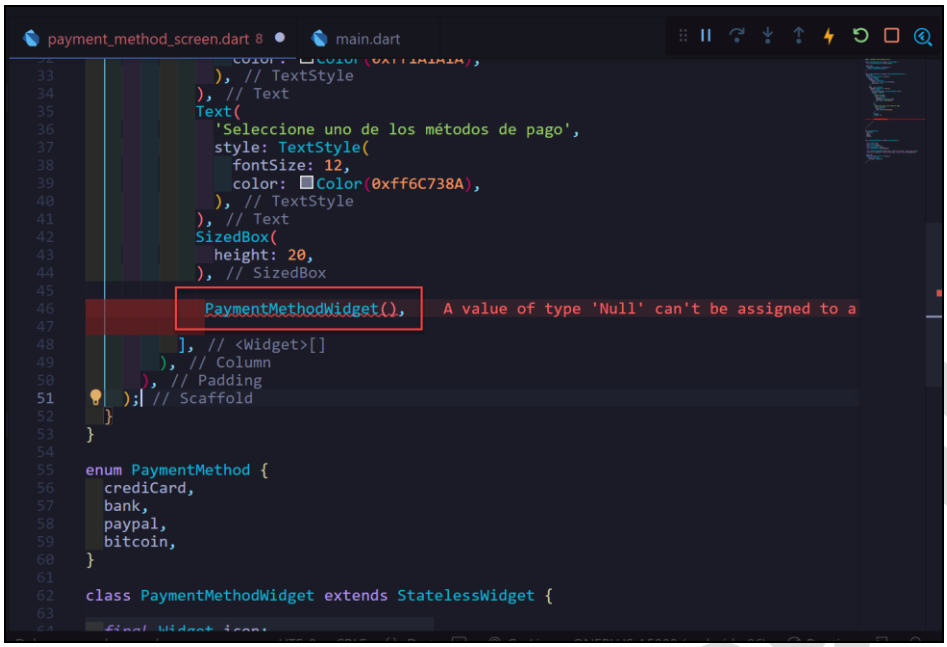
Ponemos el cursor en el constructor **PaymentMethodWidget**, de la ventana emergente **seleccionamos** la opción **Add final field formal parameters**.



18. A continuación, se muestra el **constructor resultante** del proceso.

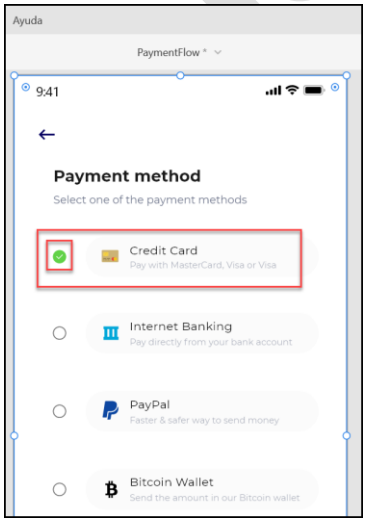


19. Ahora **el error** se genera en la invocación del **constructor** de la clase **PaymentMethodWidget**, puesto que ya **no existe** un **constructor con 0 parámetros**.

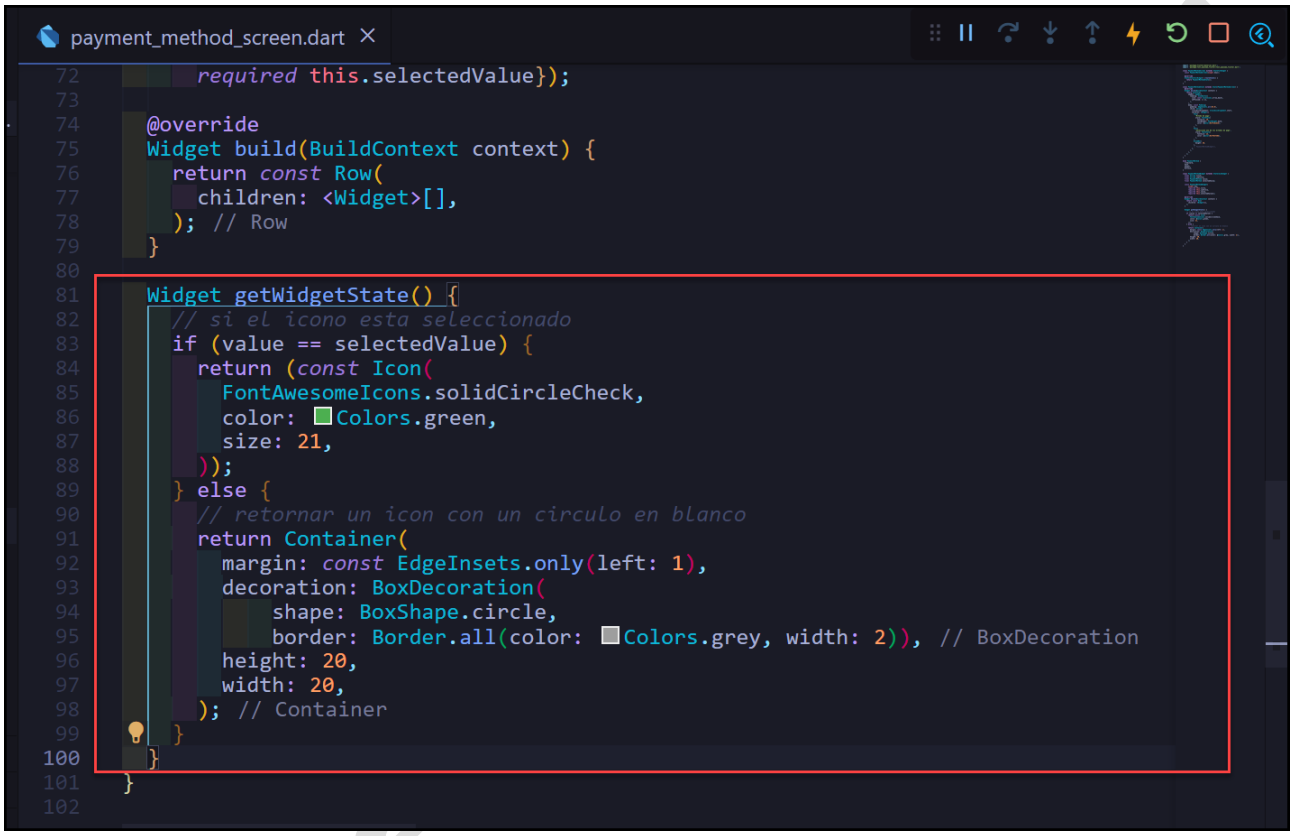


20. Este error se va a corregir más adelante, en los siguientes puntos de la práctica.

21. Ahora que ya tenemos el **widget personalizado** para representar **cada método de pago**, visualice nuevamente el **mockup** para continuar creando la UI de los **métodos de pago**. Lo que se muestra en el **mockup** puede ser representada por una **fila** (Row), lo **primero** que se muestra en la **fila** es un **icono** que indica si el **método de pago** está o no **seleccionado** (parte izquierda, icono verde).



22. Para esto en el **payment_method_screen.dart** cree el **método** llamado **getWidgetState**, este se va a utilizar representar cada **método de pago**, el método debe regresar el **estado** (con un widget) del **método de pago seleccionado** (un círculo **verde** si esta seleccionado, o circulo en **blanco** si no está seleccionado), es decir si está o no **seleccionado** se regresa el icono del método de pago. En la clase **PaymentMethodWidget**, luego del método **build**, como se muestra a continuación.



El método hace lo siguiente: si el **icono** que representa la forma de pago esta **seleccionado** se regresa un widget Icon (**icono**) del paquete **font awesome** de **color verde**. Caso contrario se regresa un widget Container (**contenedor**) al cual se le da una decoración con forma de un **círculo en blanco** que tiene la apariencia de un botón de Radio sin seleccionar.

Nota	• Note que para poder utilizar un **icono** de **Font Awesome** se debe importar al **inicio** del archivo el **paquete** de Font Awesome. Esto se configuro anteriormente en el archivo **pubspec.yaml**.

- Paquete de **Font Awesome** agregado al inicio del archivo:

```dart
import 'package:flutter/material.dart';
import 'package:font_awesome_flutter/font_awesome_flutter.dart';

class PaymentMethodScreen extends StatefulWidget {
  const PaymentMethodScreen({super.key});
```

23. En la clase **PaymentMethodWidget** llame al método **getWidgetState** creado anteriormente, dentro del método **build**, este de acuerdo con el mockup es el **primer elemento** de la fila **fila** (**Row**) que representa un método de pago (la figura se muestra en el punto 21).

```dart
  paypal,
  bitcoin,
}

You, 18 seconds ago | 1 author (You)
class PaymentMethodWidget extends StatelessWidget {
  final String title;
  final String subTitle;
  final PaymentMethod value;
  final PaymentMethod selectedValue;

  const PaymentMethodWidget(
      {super.key,
      required this.title,
      required this.subTitle,
      required this.value,
      required this.selectedValue});

  @override
  Widget build(BuildContext context) {

    return Row(      You, 14 seconds ago • Uncommitted changes
      children: <Widget>[
        getWidgetState(),
      ], // <Widget>[]
    ); // Row

  }

  Widget getWidgetState() {
    // si el icono esta seleccionado (retornar icono verde)
    if (value == selectedValue) {
      return (const Icon(
```

Nota	• Para que no genere errores el código anterior, fíjese que **se ha quitado** la palabra reservada **const** del **return** señalado en la figura anterior. • Analice por qué no es necesario eliminar el **const** del código anterior.

24. En la clase **_PaymentMethodState** del mismo archivo, agregue la variable llamada **paymentSelected** e inicialice esta variable con un valor inicial para el método de pago, en este caso el valor establecido es **creditCard** que representa el método de pago con tarjeta de crédito, esta variable a nivel de clase **permite** conocer (guardar) el valor **seleccionado** en el método de pago.

payment_method_screen.dart M | test.dart U

```dart
import 'package:flutter/material.dart';
import 'package:payment_flow/ui/components/payment_type.dart';

class PaymentMethodScreen extends StatefulWidget {
  const PaymentMethodScreen({super.key});

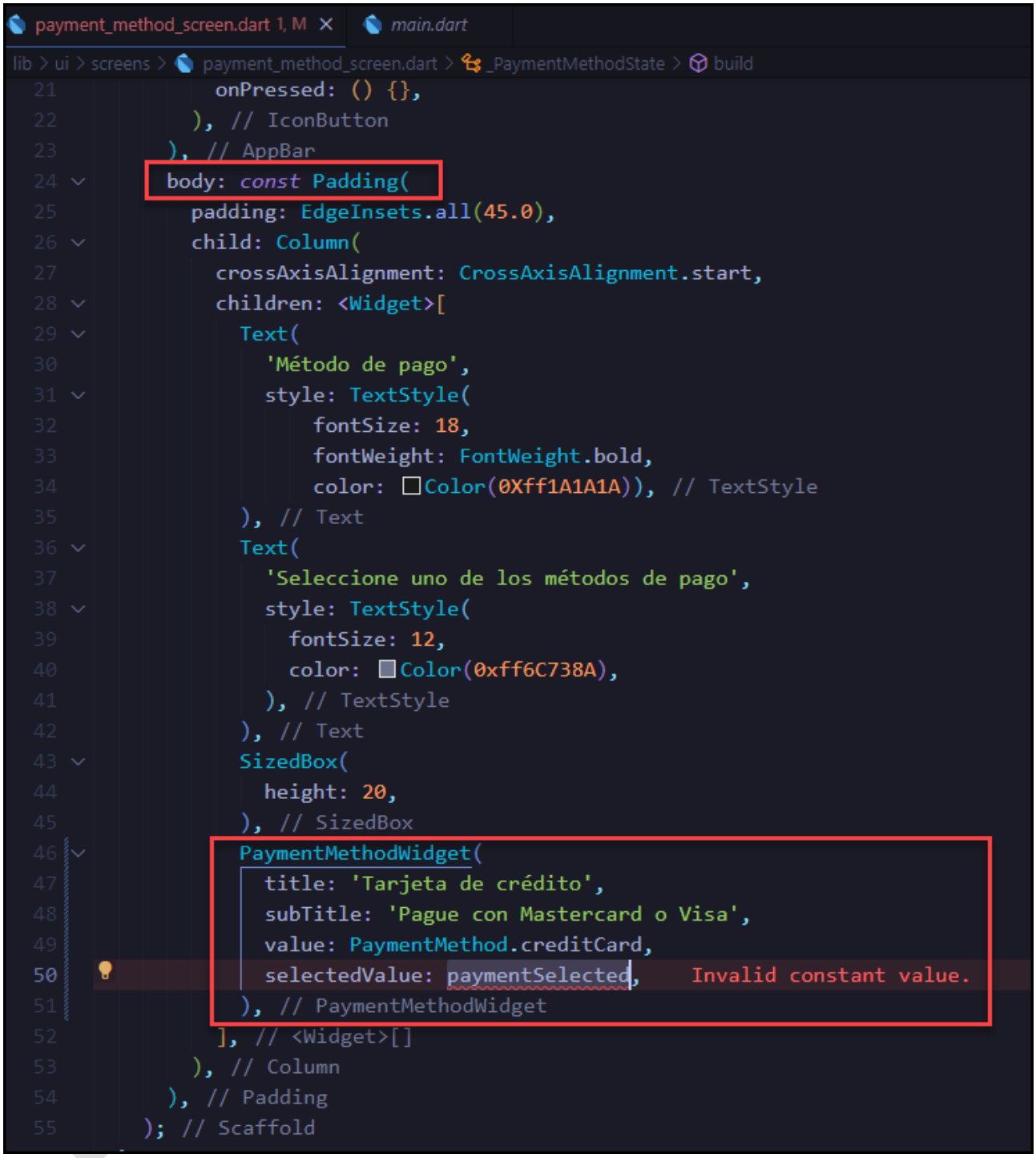
  @override
  State<StatefulWidget> createState() {
    return _PaymentMethodState();
  }
}

class _PaymentMethodState extends State<PaymentMethodScreen> {

  PaymentMethod paymentSelected = PaymentMethod.crediCard;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        leading: IconButton(
          icon: const Icon(Icons.arrow_back),
          onPressed: () {},
        ), // IconButton
      ), // AppBar
    ); // Scaffold
  }
```

25. Para poder **ejecutar** y **probar** el proyecto **reemplace** la llamada anterior del **constructor** sin parámetros de la clase **PaymentMethodWidget** con el nuevo constructor con **4 parámetros**. Recuerde que internamente por medio del método **getWidgetState** se **regresa** el icono (**verde**) o el icono (en **blanco**) en el widget **Row**. Esto lo hacemos en el método **build** de la clase **_PaymentMethodState**. Por lo tanto, cree la primera fila del UI para el método de pago con el siguiente código.



- En los 2 primeros parámetros (**title** y **subtitle**) del método **PaymentMethodWidget** usamos **valores** de acuerdo con el **mockup**, aunque no se va a visualizar porque **aún no se crea** la interfaz para mostrar estos valores.
- Para el **tercer** y **cuarto** parámetro usamos la enumeración (**PaymentMethod**) con **valores iguales** (puesto que la variable de clase **paymentSelected** tiene el valor por

defecto en **creditCard**) para que se ejecute la cláusula **if** del método **getWidgetState** que regresa un **icono en color verde**.

26. Como se puede visualizar en la **figura anterior**, hay un **error** en el parámetro por nombre **selectedValue**, el error nos dice que no es un **valor constante válido**. Esto es porque el widget **Padding** está marcado como **constante**, y ya **no puede ser constante** porque la variable **selectedValue** cambia en el proyecto, por lo tanto, debemos eliminar el **const** del **Padding** y agregamos el **const** donde sea **necesario**, por lo tanto, **lo eliminamos**. En el código siguiente se **resalta** las líneas donde se hizo **cambios**, esto se muestra a continuación.

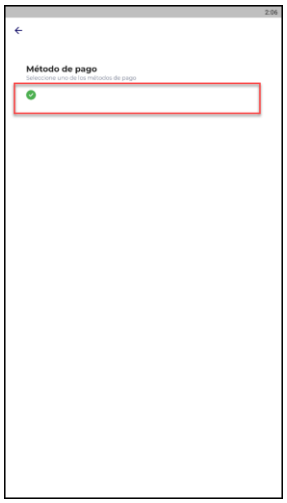
Cambios realizados para que ejecute el proyecto.

payment_method_screen.dart M × main.dart

lib > ui > screens > payment_method_screen.dart > _PaymentMethodState > build

```
21              onPressed: () {},
22            ), // IconButton
23          ), // AppBar
24          body: Padding(
25            padding: const EdgeInsets.all(45.0),      You, 15 seconds ago • Uncommitted changes
26            child: Column(
27              crossAxisAlignment: CrossAxisAlignment.start,
28              children: <Widget>[
29                const Text(
30                  'Método de pago',
31                  style: TextStyle(
32                      fontSize: 18,
33                      fontWeight: FontWeight.bold,
34                      color: Color(0Xff1A1A1A)), // TextStyle
35                ), // Text
36                const Text(
37                  'Seleccione uno de los métodos de pago',
38                  style: TextStyle(
39                    fontSize: 12,
40                    color: Color(0xff6C738A),
41                  ), // TextStyle
42                ), // Text
43                const SizedBox(
44                  height: 20,
45                ), // SizedBox
46                PaymentMethodWidget(
47                  title: 'Tarjeta de crédito',
48                  subTitle: 'Pague con Mastercard o Visa',
49                  value: PaymentMethod.creditCard,
50                  selectedValue: paymentSelected,
51                ), // PaymentMethodWidget
52              ], // <Widget>[]
53            ), // Column
54          ), // Padding
55        ); // Scaffold
56      }
```

27. **Ejecute** el proyecto para comprobar que todo funcione correctamente.



28. En la clase **_PaymentMethodState** donde se llama al método **PaymentMethodWidget**, cambie el valor de la **enumeración** (**PaymentMethod**) **value**, en este caso para el valor **paypal** para que tenga un **valor diferente** y por lo tanto se ejecute la **cláusula else** del método **getWidgetState** y se muestre y regrese un **icono en blanco**. Por ejemplo.

```
main.dart | payment_method_screen.dart

51            const SizedBox(
52              height: 20,
53            ), // SizedBox
54
55            //PaymentMethodWidget(),
56
57            PaymentMethodWidget(
58              title: 'Tarjeta de Crédito',
59              subTitle: 'Pague con Mastercard o Visa',
60              value: PaymentMethod.paypal,
61              selectedValue: paymentSelected,
62            ), // PaymentMethodWidget
63          ], // <Widget>[]
64        ), // Column
65      ), // Padding
66    ); // Scaffold
67  }
68  }
69
70  enum PaymentMethod {
71    crediCard,
72    bank,
73    paypal,
74    bitcoin,
75  }
```

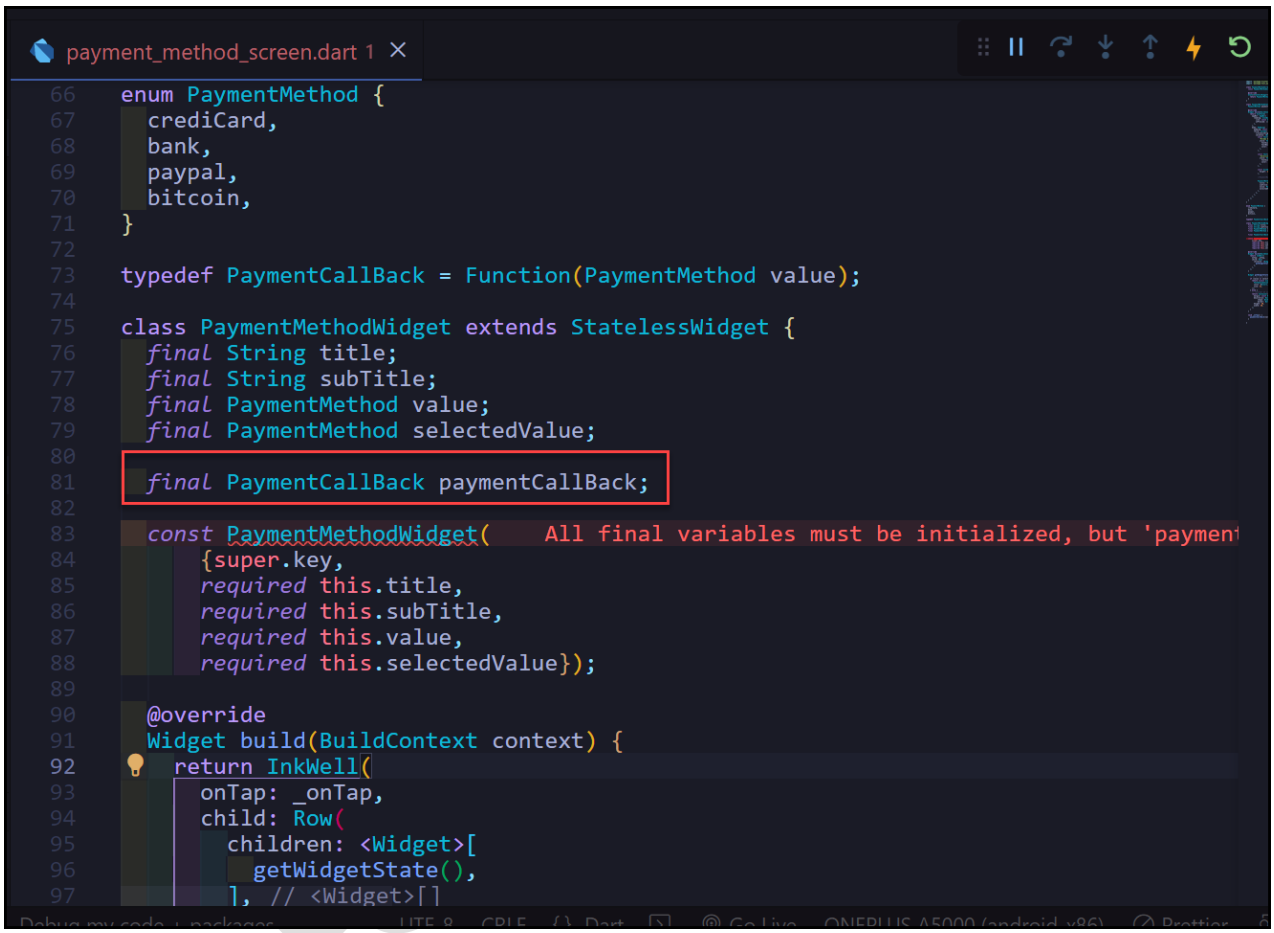
29. **Ejecute** el proyecto para comprobar que todo funcione correctamente.



30. Luego de la enumeración, agregue un **listener** (manejador), es decir un **método** que responda al **cambio de estado** cuando se **haga un clic** en el botón de tipo **Radio** que establece el tipo de **método de pago** seleccionado. Lo creamos como un tipo de dato (**typedef**) llamado **PaymentCallBack** de tipo **function** y le pasamos como parámetro el tipo de método de pago, esto es de tipo **PaymentMethod**.

```
main.dart    payment_method_screen.dart
62      ); // Scaffold
63    }
64  }
65
66  enum PaymentMethod {
67    crediCard,
68    bank,
69    paypal,
70    bitcoin,
71  }
72
73  typedef PaymentCallBack = Function(PaymentMethod value);
74
75
76  class PaymentMethodWidget extends StatelessWidget {
77    final String title;
78    final String subTitle;
79    final PaymentMethod value;
80    final PaymentMethod selectedValue;
81
82    const PaymentMethodWidget(
83        {super.key,
84        required this.title,
85        required this.subTitle,
86        required this.value,
87        required this.selectedValue});
88
```

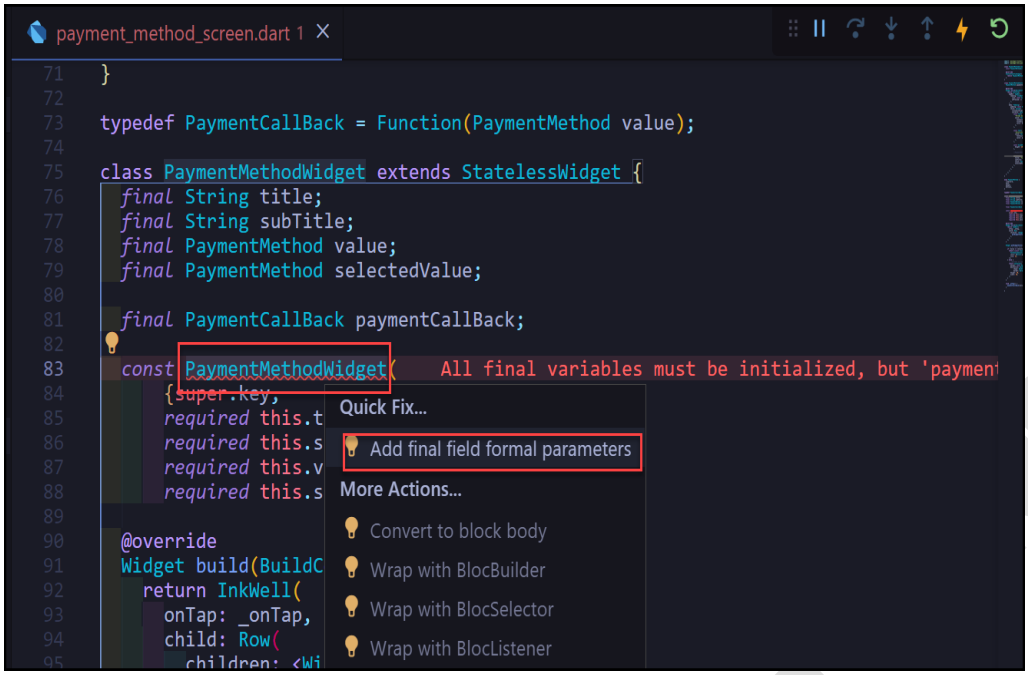
31. Una vez creado el tipo de dato **PaymentCallBack** en el punto anterior, Defina una variable de este tipo en el widget **PaymentMethodWidget** llamada **paymentCallBack**, este se utilizará para llamar a la función para que se **ejecute** al ocurrir el evento **clic** (**onTap**) en el widget de tipo **InWell** (que se creará en el siguiente punto), esto va a permitir conocer el tipo de pago **seleccionado**. El código se muestra a continuación.



La función que llama al **callback** tiene como **parámetro** una variable de tipo **PaymentMethod** (enumeración) que es la variable que va a cambiar al hacer un clic en el botón de **Radio** (Realmente un **Container**).

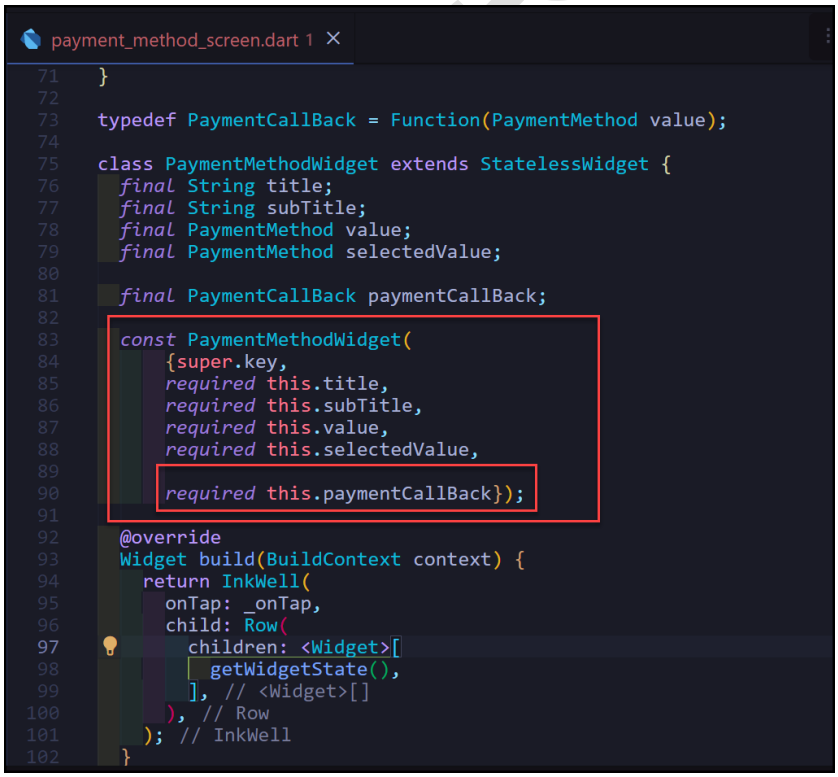
32. Como se puede visualizar en la figura anterior ahora el **constructor** requiere que la **nueva variable** creada (**paymentCallBack**) sea inicializada en el constructor. Para esto nos ayudamos de **Visual Studio Code**. Proceda de la siguiente forma, **ponga el mouse** sobre el **constructor** de la clase **PaymentMethodWidget**, luego presione las teclas **CONTROL + PUNTO** (CONTROL + .). De la ventana emergente **seleccione** la opción **Add final field formal parameters**. Esto se muestra a continuación.

Seleccione la opción indicada.



33. Esto modifica el **constructor**, añadiendo el parámetro **paymentCallBack** al final de todos los parámetros. El constructor es el siguiente luego de este cambio.

Constructor modificado.




```
}

typedef PaymentCallBack = Function(PaymentMethod value);

class PaymentMethodWidget extends StatelessWidget {
  final String title;
  final String subTitle;
  final PaymentMethod value;
  final PaymentMethod selectedValue;

  final PaymentCallBack paymentCallBack;

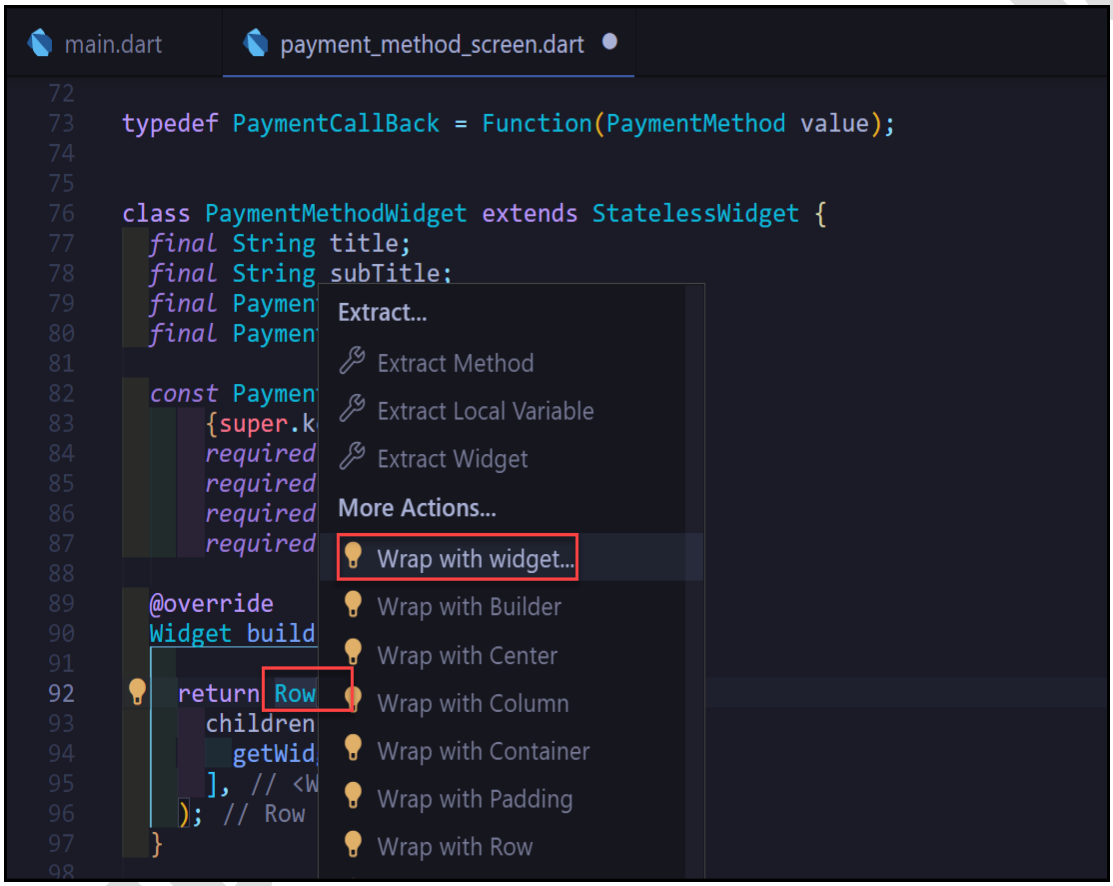
  const PaymentMethodWidget(
      {super.key,
      required this.title,
      required this.subTitle,
      required this.value,
      required this.selectedValue,

      required this.paymentCallBack});

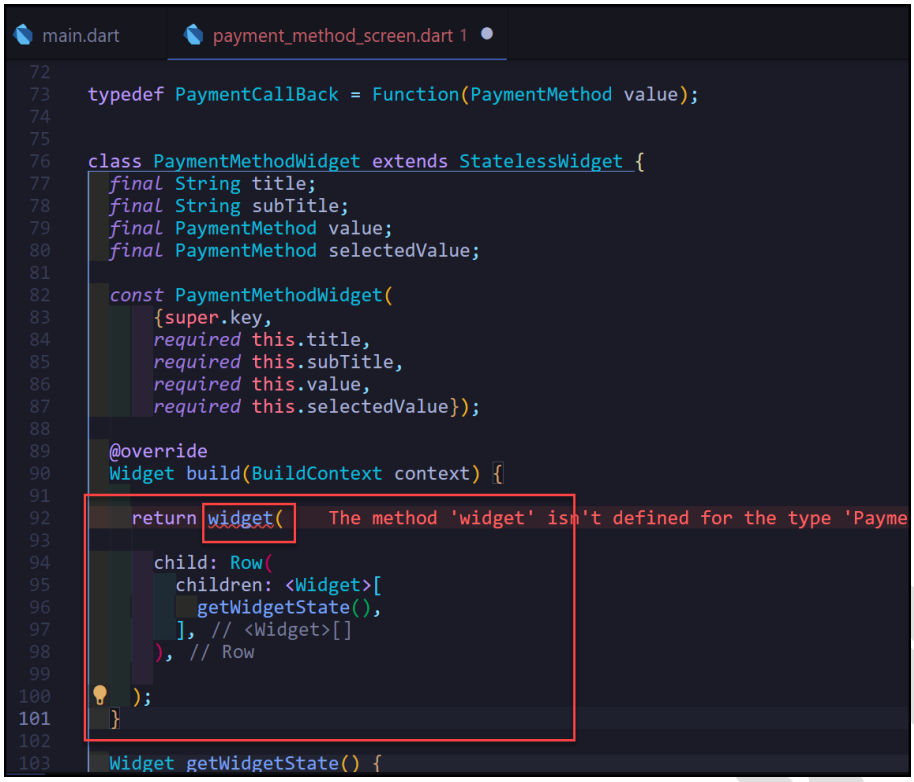
  @override
  Widget build(BuildContext context) {
    return InkWell(
      onTap: _onTap,
      child: Row(
        children: <Widget>[
          getWidgetState(),
        ], // <Widget>[]
      ), // Row
    ); // InkWell
  }
```

34. Para **ejecutar** la función creada al hacer un **clic** en el botón de **Radio** (que en realidad más adelante lo vamos a crear con un **Container** con una figura en forma de **circulo** que simule el botón de radio), es necesario que **este sea seleccionable** (clicleable), para esto lo envolvemos el **Row** dentro de un widget **InkWell**. Nos ayudamos del **Visual Studio Code** para **generar** el código necesario, por lo tanto **ponga el mouse** sobre la palabra **Row** del método **build** del widget **PaymentMethodWidget**, presione las teclas **CONTROL + PUNTO** (CONTROL + .). De la ventana emergente **seleccione** la opción **wrap with widget**.

Seleccione el **Row** y de la ventana emergente la opción indicada.



35. Esto hace que el **Row** se **envuelva** en un **widget**, con el siguiente código.



36. **Cambie** la palabra **widget** generada anteriormente por Visual Studio, con la cual está **envuelto** el **Row**, la palabra **widget** cámbielo por un widget **InkWell**, el widget **InkWell** permite definir un área **rectangular** que permite **responder a un evento**.



37. **Agregue** el parámetro con nombre llamado **onTap** del widget **InkWll** que responda al evento **clic** y en este invoque a un **método** llamado **_onTap**, este método aún **no existe**, se lo va a crear en el **próximo punto**.



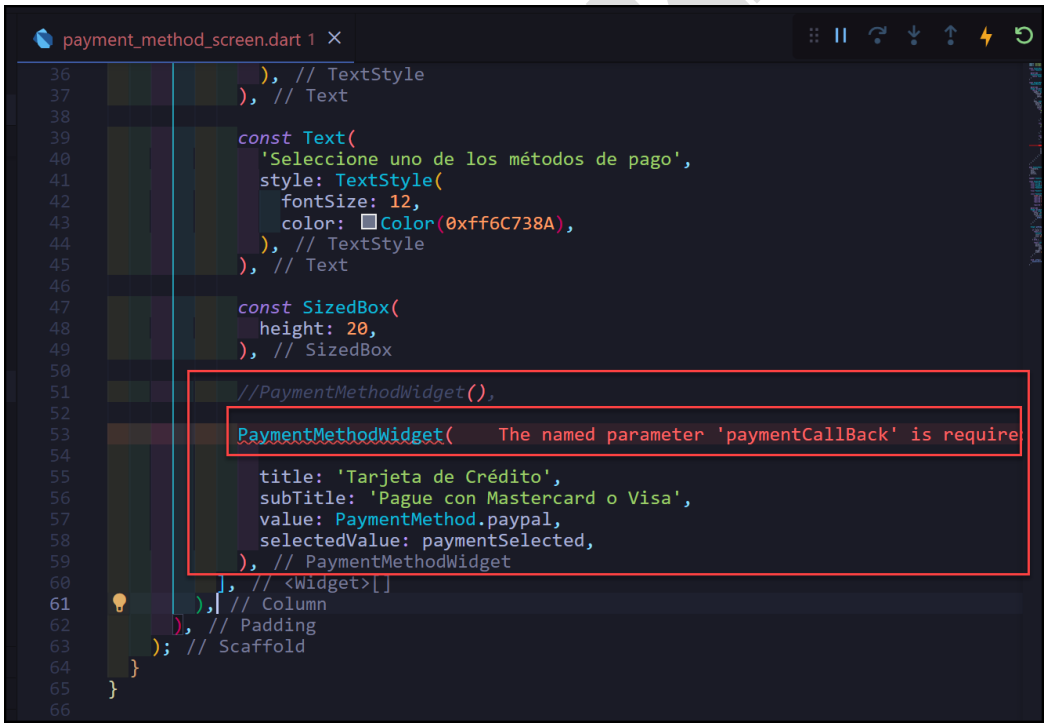
38. **Al final** de la clase **PaymentMethodWidget** defina el método **_onTap**.



39. La variable **paymentCallBack** sirve para poder **notificar** el cambio del **estado** en el widget **_PaymentMethodState**, de esta forma al cambiar el valor de este se puede llamar al método **setState** para modificar el **estado** del widget y por lo tanto cambiar el valor de la variable **selectedValue** (enumeración con los valores: crediCard, bank, paypal, bitcoin).

- Este método se ejecuta al hacer un **clic** en el botón de **Radio** (en realidad un Container).
- Este método debe regresar un **void** obligatoriamente porque responde a un evento.
- Dentro del método **_onTap**, se utiliza la variable **paymentCallBack** para llamar a la **función** que permite realizar el **callback** (**typedef PaymentCallBack = Function(PaymentMethod value);**).

40. Como ahora el **constructor** del widget **PaymentMethodWidget** tiene un nuevo **parámetro**, se genera un **error** en llamada al constructor de la clase **PaymentMethodWidget**, en el método **build** (clase **_PaymentMethodState**), esto se muestra en la figura siguiente.



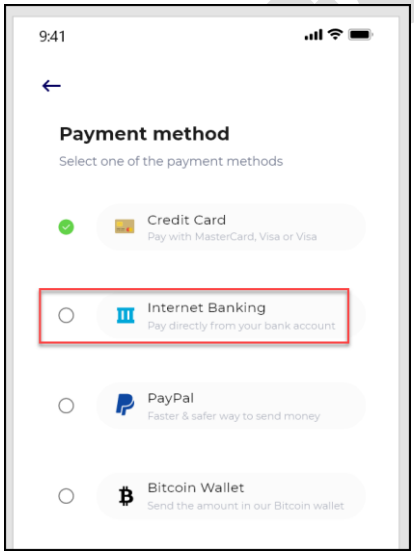
41. Para corregir el error **debemos** utilizar el parámetro **paymentCallBack** para llamar al **método** **setState** y **cambiar** el estado de la variable **paymentSelected** de la clase

_PaymentMethodState que **modifica el estado** del tipo de pago **seleccionado**. A continuación, el código para realizar esta tarea.

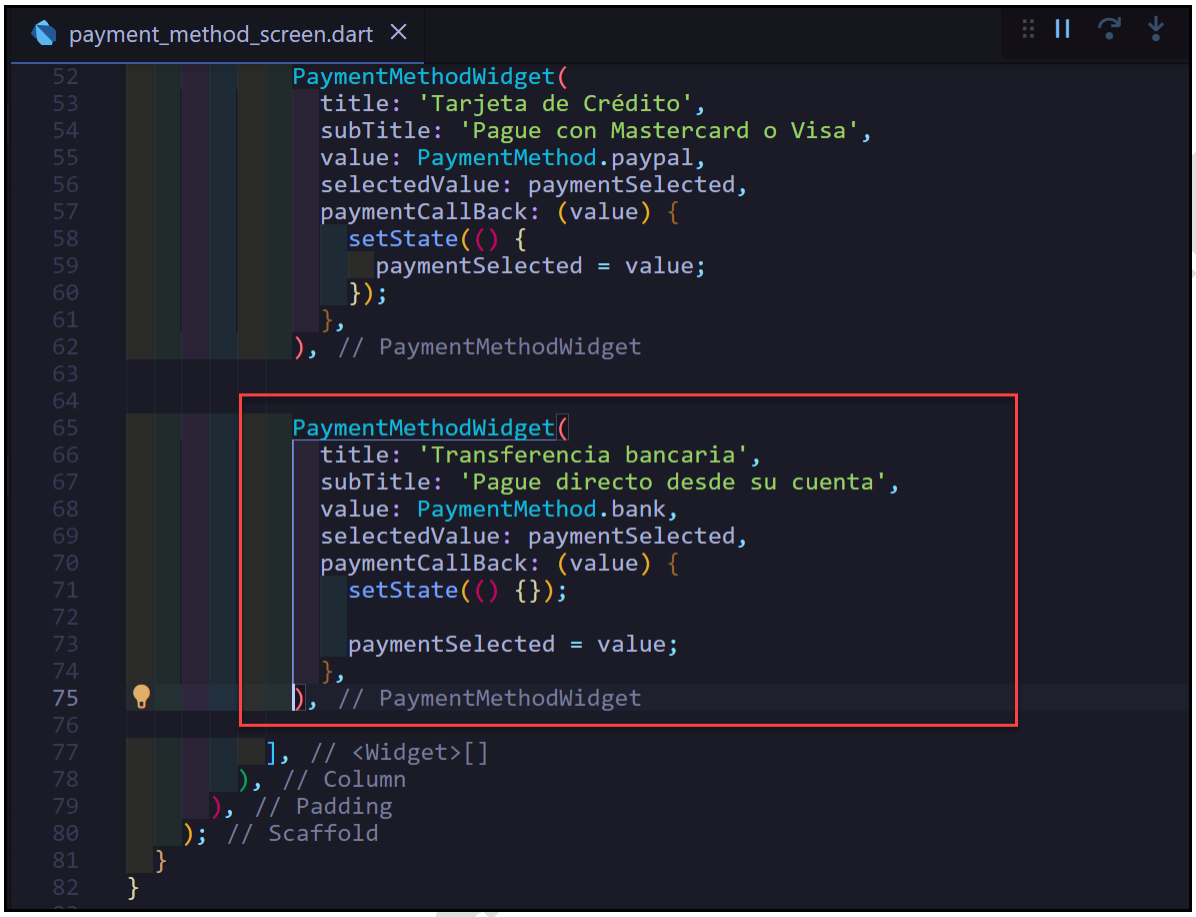
payment_method_screen.dart

```
38
39              const Text(
40                'Seleccione uno de los métodos de pago',
41                style: TextStyle(
42                  fontSize: 12,
43                  color: Color(0xff6C738A),
44                ), // TextStyle
45              ), // Text
46
47              const SizedBox(
48                height: 20,
49              ), // SizedBox
50
51              //PaymentMethodWidget(),
52
53              PaymentMethodWidget(
54                title: 'Tarjeta de Crédito',
55                subTitle: 'Pague con Mastercard o Visa',
56                value: PaymentMethod.paypal,
57                selectedValue: paymentSelected,
58
59                paymentCallBack: (value) {
60                  setState(() {
61                    paymentSelected = value;
62                  });
63                },
64
65              ), // PaymentMethodWidget
66            ], // <Widget>[]
67          ), // Column
68        ), // Padding
69      ); // Scaffold
```

42. Siguiendo con el diseño del **mockup** ahora debemos **crear** la UI para la siguiente **fila** del tipo de pago, esto es para la **Transferencia bancaria**,

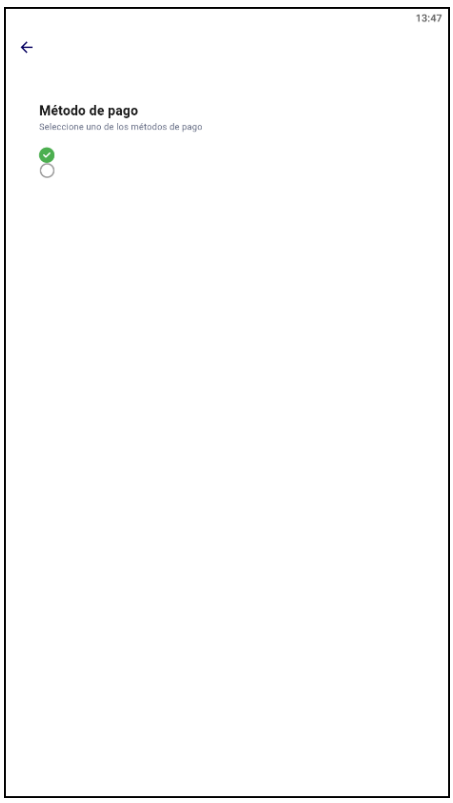


43. Puede **copiar**, **pegar** y **modificar** el código anterior. A continuación, se presenta el código.



44. **Fíjese** muy bien en la forma de usar el método **setState** para cambiar el **estado** de la variable **paymentSelected**, en el punto 41 esta línea **está dentro** del método **setState**, en el punto 43 esta línea se encuentra **fuera** del método **setState**. Las dos formas de usar el método **setState** son **equivalentes**, depende del desarrollador cual **utilizar**.

45. **Ejecute** y pruebe el **proyecto** para verificar que todo funcione correctamente.



Lecturas útiles para revisar y profundizar en los temas abordados en esta práctica.

Row class

https://api.flutter.dev/flutter/widgets/Row-class.html

SizedBox class

https://api.flutter.dev/flutter/widgets/SizedBox-class.html

Container class

https://api.flutter.dev/flutter/widgets/Container-class.html

BoxDecoration class

https://api.flutter.dev/flutter/painting/BoxDecoration-class.html

InkWell class

https://api.flutter.dev/flutter/material/InkWell-class.html

The semantics of typedef in dart

https://medium.com/embrace-it/the-semantics-of-typedef-in-dart-783ebeb833a0

flutterClay Aldás